

@intentsolutions/core: Operator-Grade System Analysis

Generated: 2026-05-20 Version: git tag v0.1.0 (commit c9b9f75); audit run against HEAD ac0cdec (post-v0.1.0 follow-on epics iec-E07/E10/E11/E12)

1. This System in 5 Minutes

@intentsolutions/core is the **canonical contracts kernel** for the Intent Eval Platform — a single npm package that defines, in three coordinated forms, every shared shape that flows between the five platform repos. The three forms are: TypeScript interfaces (`src/entities/`, `src/predicates/`, `src/primitives.ts`), JSON Schema draft-2020-12 documents (`schemas/v1/`), and Zod runtime validators (`src/validators/v1/`). Each form mirrors the same 13 entities and the same one fully-normative in-toto predicate body (`gate-result/v1`). The package compiles to ESM in `dist/`, exports tree-shakable subpaths, ships with **zero runtime dependencies** at the main entry (zod is loaded only by consumers who import from `./validators/v1`), and is published with sigstore-keyless provenance through npm's OIDC flow.

The kernel exists because before it, the same 13 entities were redefined locally in `audit-harness`, `j-rig-skill-binary-eval`, and `intent-rollout-gate` — each with slightly different field names, missing fields, and mutually incompatible validators. The lab's earlier `gate-result/v1` v0.1.0-draft schema had drifted from Blueprint B § 7.4, and DR-010's "unification thesis" (every validator emits an Evidence Bundle) was unenforceable. This package replaces that drift with one source of truth: the kernel **defines** the canonical contracts; everyone else **consumes** them.

It is read by humans (the entity files are the readable form of Blueprint B § 2), imported by TypeScript consumers (`import type { EvalSpec } from '@intentsolutions/core'`), pulled by polyglot consumers (`schemas/v1/*.schema.json` is the language-neutral wire format), and validated at runtime by anyone who needs structural enforcement (`GateResultV1Schema.parse(payload)`). The kernel runs on consumer machines and in consumer CI; it has no servers of its own, no DB, no scheduler, no queue, no LLM provider adapter, and no execution loop. That absence is enforced architecturally — a 4-axis boundary checker (`scripts/check-boundaries.ts`) reads `FORBIDDEN.md` + `ALLOWLIST.md` and blocks pre-commit + PR if anything that smells like a runtime tries to land.

Current state (2026-05-20): v0.1.0 shipped on 2026-05-17 from CI with sigstore provenance, closing five epics (`iec-E01`, `iec-E02`, `iec-E03`, `iec-E04`, `iec-E09`). Since then, three follow-on epics have landed on `main`: `iec-E10` (per-repo blueprint applying Blueprint C), `iec-E11` (4-axis boundary enforcement + CI workflow + `ALLOWLIST/FORBIDDEN/CODEOWNERS` narrowing), and `iec-E12` (testing SOP + CI/CD ratify-and-close AAR + boundary-check required status). The latest commit (`ac0cdec`, `iec-E07`) adds an api-extractor SemVer regression gate that diffs the public surface against a committed golden snapshot. The 9-step CI gate chain is green; the boundary check is green; coverage is 100% line/branch/function/statement; the architecture rules emit 0 violations across 8 forbidden dep-cruiser entries.

The biggest **risk** is consumer-side adoption inertia. The kernel was published two weeks before this audit but the three downstream sibling repos (`audit-harness`, `j-rig-skill-binary-eval`, `intent-rollout-gate`) have not yet migrated their local schemas onto it. Each day they don't, the unification thesis weakens — sibling repos accumulate divergent local types, and the canonical-shapes guarantee depends on the migration actually happening. The second-biggest risk is the

JSON-Schema-vs-Zod twin-source maintenance burden: the two sources can drift, and a six-axis defense (codegen reference output in `_generated/`, hand-authored canonical validators in `src/validators/v1/`, schema-validation tests + Zod-validation tests + ERD-walk integration tests + tsd type tests) is what keeps them aligned. If any of those defenses lapse, the divergence will be silent.

The third-biggest risk is deferred security posture: while the publish flow uses npm provenance (sigstore-keyless via GitHub OIDC) and consumers can run `npm audit signatures`, several pieces of the broader DSSE/Rekor story are still deferral beads — `signing_mode` field enforcement in DSSE (Security C-3), Rekor pre-flight validation (Security C-2), and CISO-binding DNSSEC + CAA pinning on `evals.intentsolutions.io` before first production-Rekor unlock. Those are tracked but not yet closed.

2. Executive Summary

What It Does

`@intentsolutions/core` is a published npm library (`@intentsolutions/core@0.1.0`, Apache-2.0, ESM, Node 20+). Its single responsibility is to publish a frozen, versioned, sigstore-signed set of canonical contracts for the Intent Eval Platform. The contracts come in three forms — TypeScript interfaces, JSON Schemas, and Zod runtime validators — and they cover three things: the 13 canonical platform entities defined in Blueprint B § 2 (`EvalSpec`, `EvalRun`, `MatcherMap`, `EvidenceBundle`, `JudgeDecision`, `RuntimeReceipt`, `RegressionPack`, `RolloutGate`, `SkillSnapshot`, `SessionTrace`, `ToolInvocation`, `CostRecord`, `FailureTaxonomy`), the one fully-normative in-toto predicate body (`gate-result/v1`) per Blueprint B § 7.4, and ten branded primitive types (`Uuidv7`, `Sha256`, `Sha256Prefixed`, `Rfc3339`, `SemVer`, `KebabSlug`, `MicroUsd`, `StorageKey`, `OtelSpanId`, `ActorIdentity`).

Implementation status: **fully implemented for v0.1**. The five epics that constitute v0.1.0 are closed (`iec-E01` repo scaffold; `iec-E02` 13 TS entities + `gate-result/v1` + IS Testing SOP install; `iec-E03` JSON Schemas; `iec-E04` Zod runtime validators; `iec-E09` NPM publishing with sigstore provenance). Three follow-on epics have also landed: `iec-E10` (per-repo blueprint at `000-docs/002-AT-ARCH-...md`), `iec-E11` (4-axis boundary enforcement — `FORBIDDEN.md`, `ALLOWLIST.md`, `scripts/check-boundaries.ts`, `.github/workflows/boundary-check.yml`), and `iec-E12` (testing SOP + CI/CD bootstrap AAR + boundary-check required status). The latest commit (`ac0cdec`) closes `iec-E07` (api-extractor SemVer regression gate against a committed golden snapshot). All this exists on `main`; the published v0.1.0 reflects the state up through `iec-E09`, and v0.1.1 (when it ships) will reflect the additions.

Technical foundation: TypeScript 5.7 with every available strict flag enabled (`noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`, `noImplicitOverride`, `noPropertyAccessFromIndexSignature`, `verbatimModuleSyntax`); pnpm 9 workspace (single package, not a monorepo); vitest 2 for tests; tsd for negative-position type tests; dependency-cruiser 17 for architecture rule enforcement; eslint 9 flat config with typed linting; husky for pre-commit gating; api-extractor 7 for SemVer golden-snapshot diffing; sigstore via npm OIDC for publish provenance. The `dist/` artifact is ~280 KB unpacked, 159 files, no runtime deps in the main entry, `zod` ^4.4.3 listed as a peer/runtime dep used only by the `validators/v1/*` subpath.

Key risks (audit-time, ordered by severity): (1) downstream sibling repos have not yet migrated — `audit-harness` (tracked at `iah-E02`), `j-rig-skill-binary-eval` (`iaj-E02`), `intent-rollout-gate` (no bead claimed); (2) JSON Schema + Zod twin-source maintenance burden; (3) deferred DSSE/Rekor posture (`bd_000-projects-k0fj` `tenant_id` reservation, Security C-2 Rekor pre-flight, Security C-3 `signing_mode` in DSSE — all unresolved); (4) the lab's `iel-link-schemas-to-kernel` bead is still open, meaning the lab's own canonical schemas still need to be repointed at the kernel.

Operational Status

Table 1.

Environment	Status	Uptime Target	Release Cadence	Last Deploy
Production (npm registry)	LIVE — <code>Contentful</code> published with <code>sigstore</code> provenance	99.9% (inherits <code>no Intent Solutions</code> own-service component)	Tag-driven; expected ~monthly through <code>v0.x</code> , then quarterly	2026-05-17
Staging	N/A — no staging registry; the npm registry is the only publish target	N/A	N/A	N/A
Local Dev	<code>pnpm install && pnpm run check</code> produces green 9-step gate chain	N/A	Pre-commit hook + push hook gate every commit	Continuous

Note: this is a **library**, not a service. There is no running process, no server, no scheduled job, no queue worker. Operational concerns live entirely in (a) the npm registry's availability

for downloads, (b) the release CI workflow's correctness when tag-triggered, and (c) downstream consumer adoption.

Technology Stack

Table 2.

Category	Technology	Version	Purpose
Language	TypeScript	<code>^5.7.2</code>	Source language; all strict flags enabled (<code>tsconfig.json:9-26</code>)
Module system	ESM (NodeNext)	n/a	<code>"type": "module"</code> in <code>package.json:26</code> ; <code>.js</code> extensions in relative imports per NodeNext resolution
Runtime target	Node.js	<code>>=20.0.0</code> (<code>.nvmrc</code> pins Node 22 for dev)	<code>package.json:52</code>
Package manager	pnpm	<code>>=9.0.0</code> (pinned <code>9.15.0</code> via <code>packageManager</code> field)	<code>package.json:55</code>
Runtime dep (kernel)	(none)	n/a	Main entry has zero runtime dependencies
Runtime dep (validators subpath)	zod	<code>^4.4.3</code>	<code>package.json:114</code> ; required only by <code>src/validators/v1/*</code>
Schema authoring	JSON Schema draft 2020-12	n/a	<code>schemas/v1/*.schema.json</code>
Codegen reference	json-schema-to-zod	<code>^2.8.1</code> (devDep)	Produces <code>_generated/*.ts</code> reference; canonical Zod validators are hand-authored
Schema validation (tests)	ajv + ajv-formats	<code>^8.20.0</code> / <code>^3.0.1</code> (devDep)	Validates the JSON Schemas against fixtures

Category	Technology	Version	Purpose
Test runner	vitest	^2.1.8	Runtime + type-level tests via <code>expectTypeOf</code>
Type-level tests	tsd	^0.33.0	Second-opinion negative type tests against the published <code>dist/</code>
Linters	eslint + typescript-eslint	^9.17.0 / ^8.19.0	Flat config (<code>eslint.config.js</code>), typed-linting via <code>projectService</code>
Formatter	prettier	^3.4.2	<code>.prettierrc.json</code>
Architecture rules	dependency-cruiser	^17.4.0 (devDep)	8 forbidden rules in <code>.dependency-cruiser.cjs</code> , hash-pinned
API surface diff	@microsoft/api-extractor	^7.58.7	Golden snapshot at <code>api/intentsolutions-core.api.m</code> , drift gate in CI
Architectural enforcement (project)	@intentsolutions/audit-harness	^0.1.0 (devDep)	escape-scan, arch-check, harness-hash, gherkin-lint, CRAP, bias-count
Hooks	husky	^9.1.7	<code>.husky/pre-commit</code> runs escape-scan + boundaries + lint-staged
Lint-staged	lint-staged	^17.0.5	Per-file linting in pre-commit
Coverage	@vitest/coverage-v8	^2.1.8	V8-instrumented; 100% floor enforced in <code>vitest.config.ts</code>
CI	GitHub Actions	n/a	<code>.github/workflows/{ci,boundary</code>
Publish provenance	npm OIDC + sigstore	n/a	<code>pnpm publish --provenance from release.yml:95</code>

Category	Technology	Version	Purpose
License	Apache-2.0	n/a	LICENSE; Apache for the kernel so every downstream (commercial + OSS + internal) can depend

3. Architecture

Stack (Detailed)

For each technology: what it does AND why it was chosen over alternatives.

Table 3.

Layer	Technology	Version	Purpose	Why This
Source	TypeScript (strict + every additional flag)	^5.7.2	Author entity interfaces, branded primitives, state-machine transition maps, and predicate body types	Two reasons. (1) DR-010 Q2 binds the platform to “TS-primary signing surfaces; Python permitted ML internals” — every signed contract has to be authored in TypeScript first. (2) <pre>verbatimModuleSyntax + noUncheckedIndexedAccess + exactOptionalPropertyTypes</pre> together encode invariants in the type system that would otherwise require runtime checks (cf. <code>tsconfig.json:21-26</code>). Plain JavaScript was never on the table; less-strict TS would mean weaker compile-time guarantees on the contracts everyone else trusts.

Layer	Technology	Version	Purpose	Why This
Wire format	JSON Schema draft 2020-12	n/a	Language-neutral representation of the same 13 entities + the predicate body; \$ref cross-references via <code>_common.schema.json</code> to avoid duplication	The platform must serve non-TS consumers eventually (Python via Pydantic codegen — deferred to <code>iec-E08</code>; Rust if/when needed). JSON Schema is the lingua franca and draft 2020-12 has the modern if/then/else and \$dynamicRef constructs the gate-result/v1 advisory-severity rule needs. Alternatives considered: OpenAPI 3.1 (rejected — overkill, oriented at HTTP services we don't have); protobuf (rejected — binary-first, debugging predicate bodies in JSON is a feature, not a bug).

Layer	Technology	Version	Purpose	Why This
Runtime validation	Zod	^4.4.3	Opt-in runtime parsers per entity + the predicate; tree-shakable subpath exports so types-only consumers pay zero zod bundle cost	zod is the only runtime npm dep allowed in the kernel (ALLOWLIST.md cap of ≤ 8, current count 1/8). Alternatives considered: io-ts (rejected — fp-ts dependency tree pulls more than needed); ajv at runtime (rejected — code is JSON-data-driven so debugging refinements is awkward); valibot (rejected — newer, smaller community, would lose <code>.brand()</code> typing fidelity); arktype (rejected — same). Zod's <code>.brand<'Uuidv7'>()</code> integrates cleanly with the TS branded primitive types in <code>src/primitives.ts</code>, which is what makes the parse-and-brand pattern work end-to-end.

Layer	Technology	Version	Purpose	Why This
Branded primitives	TypeScript phantom-type brands	(compile-only)	Distinguish <code>Uuidv7</code> from <code>Sha256Prefixed</code> at the type level even though both are strings at runtime	The pattern (<code>declare const __brand: unique symbol; type Brand<T, B> = T & { readonly [__brand]: B },</code> <code>src/primitives.ts:19-20</code>) gives type-safety on the consumer side without any runtime cost. Alternatives: nominal types via class wrappers (rejected — runtime weight, fights with JSON serialization); runtime-tagged objects (rejected — same). Brands are intentionally not constructed in <code>src/primitives.ts</code> itself; the Zod parsers in <code>src/validators/v1/_primitive</code> own the <code>parse-and-brand</code>.

Layer	Technology	Version	Purpose	Why This
Codegen reference	json-schema-to-zod	^2.8.1 (devDep only)	Run via <code>pnpm run codegen:validators</code> , output to <code>src/validators/v1/_generated</code> ; used as reference , not as canonical. The hand-authored canonical Zod validators live one directory up at <code>src/validators/v1/_schemas.ts</code> .	Naive codegen produces <code>z.any()</code> for cross-file \$refs, can't apply <code>z.discriminatedUnion</code> ; (which is what makes the inferred Zod output types structurally identical to the TS interfaces), can't model superRefine rules from <code>if/then</code> blocks. and can't pick over <code>z.union</code> (which the variant types need for narrowing). Pure-codegen would either lock to the lowest-common-denominator Zod surface (worse types, worse error messages) or require post-codegen hand-patching (manual reconciliation step every regen — drift bait). Hand-authoring with codegen-as-reference is the cleaner separation. Discipline doc at <code>src/validators/v1/_generated</code> .

Layer	Technology	Version	Purpose	Why This
Architecture rules	dependency-cruiser	^17.4.0 (devDep)	Enforce: no circular deps; <code>src/(?!validators/)</code> may not import npm packages <code>(kernel-no-runtime-deps)</code> <code>src/validators/</code> may only import zod from npm <code>(validators-only-import-zod)</code> <code>src/predicates/</code> may not import from <code>src/entities/</code> <code>(predicates-no-entities)</code> <code>state-machines/</code> is a leaf <code>(state-machines-put)</code> test files cannot be imported from non-test files <code>(no-test-imports-in-src)</code> deprecated Node core modules forbidden <code>(no-deprecated-core)</code> orphan modules surfaced as warnings <code>(no-orphans)</code>	Architecture-rule enforcement has <code>src/validators/</code> live in code that runs in CI, not in prose. <code>src/validators/</code> considered: eslint-plugin-import (too coarse for the entities rule); writing a bespoke checker on top of TS AST (too <code>state-machines/</code> dep-graph slice we need). The <code>dependency-cruiser.cjs</code> file is hash-pinned via <code>.harness-hash</code> so policy changes explicit re-pinning (cf. <code>pnpm exec audit-harness</code>). <code>(no-deprecated-core)</code> orphan modules surfaced as warnings <code>(no-orphans)</code>

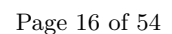
Layer	Technology	Version	Purpose	Why This
Boundary enforcement	Custom <code>scripts/check-boundaries-experimental-strip-types</code> reading <code>FORBIDDEN.md</code> + <code>ALLOWLIST.md</code>	Node	The 4-axis check what dep-cruiser doesn't: forbidden npm package names (Axis 1), forbidden source-tree paths (Axis 2), forbidden top-level directories (Axis 3), forbidden npm-keyword categories (Axis 4); also a separate URL-pattern check that blocks <code>labs.intentsolutions.io</code> as a predicate URI host (CISO binding — REFUSE, no override path)	Defense in depth: any one axis would miss classes of violation. The single-checker approach reads markdown-as-config so the same machine-readable enumeration is also the human-readable doctrine. Cf. <code>000-docs/003-AT-STND-core-rep</code> § 2 for the four-shapes-of-violation rationale.

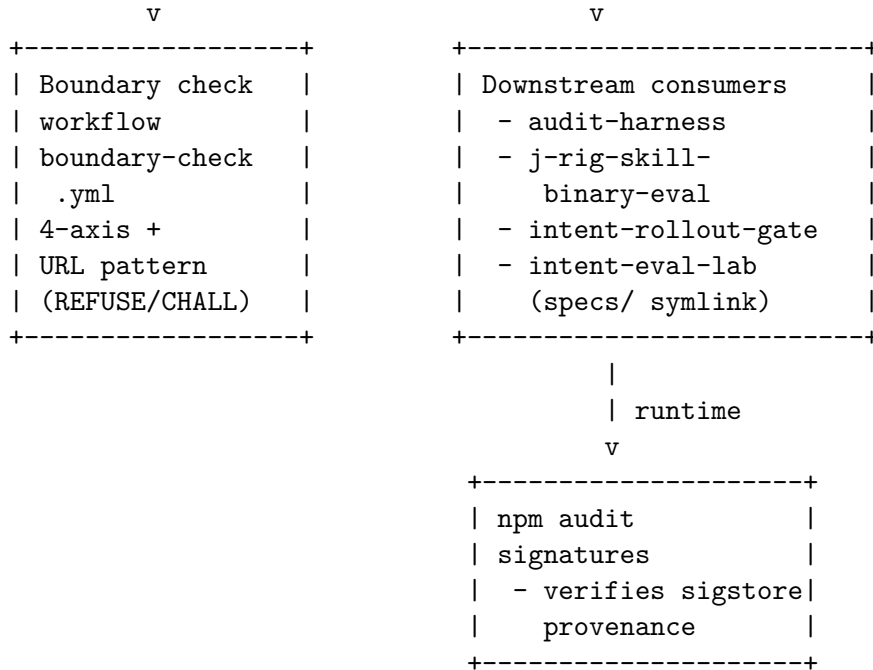
Layer	Technology	Version	Purpose	Why This
Publish provenance	npm --provenance (sigstore-keyless via GitHub OIDC)	n/a	pnpm publish --provenance --access public from .github/workflows/release.yml:95 with id-token: write permission	The publish flow runs on a tag push (v*.*.*), re-runs the full gate chain, sigstore-release.yml:95 tag-vs-package version, then pnpm publish with --provenance. The OIDC token is exchanged for a sigstore key, the tarball is signed, the provenance attestation lands at https://registry.npmjs.org/@intentsolutions/core alongside the tarball, and consumers can verify with npm audit signatures. Alternatives considered: cosign with explicit keys (rejected — key-management burden); GitHub’s own SLSA framework (rejected — not yet first-class for npm); no signing (rejected — supply-chain attack surface).

Layer	Technology	Version	Purpose	Why This
Hash-pinning	@intentsolutions/audit-harness init writing .harness-hash	^0.1.0	Policy files (.dependency-cruiser.js) are hashed; pre-commit harness:verify step refuses commits whose hashed policy doesn't match the pinned hash	If policy can be silently mutated by an LLM or an unattentive contributor, the whole boundary doctrine softens over time. Hash-pinning makes policy edits explicit + audit-trailed. Cf. .harness-hash:1 for the current pin.
api-extractor SemVer gate	@microsoft/api- extractor	^7.58.7	Public-surface golden snapshot at api/intentsolutions-disc/api.md; CI diffs current surface against snapshot; drift fails CI with instructions to bump version per per-repo blueprint § 11.1 + commit refreshed snapshot in same PR	Closes the loop on the contracts-kernel public surface cannot change without an explicit version bump and a snapshot refresh. Alternatives considered: dts- bundle-generator + diff (rejected — api-extractor's review markdown is purpose-built for this); manually-curated .d.ts declarations (rejected — too easy to drift from actual emitted surface).

System Diagram

Author / contributor laptop





Failure domains:

- **Author laptop:** pre-commit gate fails -> commit blocked. Local-only impact. Recovery: fix the failure or use `--no-verify` (discouraged; not in the documented workflow).
- **CI workflow:** lint/typecheck/test/arch/coverage/build/api-check failure -> PR check red, merge blocked. Recovery: push fix.
- **Boundary-check workflow:** 4-axis or URL violation -> PR check red, bot comment with override-bead instructions. Recovery: fix the violation or file an override bead and reference it in the PR body as `boundary-override: bd_000-projects-<id>`.
- **Release workflow:** tag-vs-package mismatch -> publish refuses. Recovery: re-tag with correct version. Sigstore/OIDC failure -> publish aborts before tarball reaches npm. Recovery: investigate Actions OIDC token issuance; do not work around.
- **npm registry:** registry outage -> consumers can't install. No mitigation owned by this repo; the registry is the only publish target by design.
- **Downstream consumer migration drift:** not a failure of this system; observable as “siblings still use local schema definitions” — tracked at the umbrella `iah-E02`, `iaj-E02`, `iel-link-schemas-to-kernel`.

The Critical Path

The critical path is **publish-and-consume**. End-to-end, step by step:

1. **Author commit** (engineer laptop). Engineer makes a change to `src/entities/EvalRun.ts` or `schemas/v1/eval-run.schema.json` or `src/validators/v1/eval-run.ts`. The pre-commit hook runs:
 - `pnpm exec audit-harness escape-scan --staged` — looks at staged diff for policy-violating patterns; refuses on hit.
 - `pnpm run boundaries` — 4-axis boundary check; refuses on URL-pattern hit, challenges on Axes 1-3 with override-bead path.

- `pnpm exec lint-staged` — runs eslint + prettier on the staged files. Failure point: any of these fails -> commit blocked.
2. **PR opened.** Two CI workflows fire in parallel:
 - `ci.yml` runs the 9-step gate chain. Failure point: any step red -> merge blocked.
 - `boundary-check.yml` runs the 4-axis check + URL check. Failure point: any axis flagged -> PR comment posted with override instructions; PR blocked.
 3. **Merge to main.** Once merged, the same CI workflow runs again on `main`. Failure point: same as #2.
 4. **Tag push.** Engineer pushes `git tag -a vX.Y.Z + git push origin vX.Y.Z` after bumping `package.json#version` and updating `CHANGELOG.md`.
 5. **Release workflow** (`release.yml`) fires on the tag. It:
 - Checks out with full history (`fetch-depth: 0`).
 - Verifies the tag (`vX.Y.Z`) matches `package.json#version` (defends against tag-vs-package drift). Failure point: mismatch -> abort before publish.
 - Re-runs the full gate chain (`harness:verify`, lint, typecheck, arch, test, coverage, build, test:types, dist-verify).
 - Runs `pnpm publish --no-git-checks --provenance --access public` with `NODE_AUTH_TOKEN` from secrets and `id-token: write` for the sigstore OIDC exchange. Failure point: provenance generation fails (e.g., OIDC token issuance fails) -> publish aborts before tarball reaches npm; recovery is to investigate Actions OIDC config.
 6. **npm registry** indexes the tarball + sigstore provenance attestation. Failure point: registry outage. No mitigation owned by this repo.
 7. **Consumer install.** Downstream repo (e.g., `audit-harness`) runs `pnpm add @intentsolutions/core` or bumps the existing version. The package's main entry resolves to `dist/index.js` (zero deps); the validators subpath resolves to `dist/validators/v1/<entity>.js` (zod peer dep).
 8. **Consumer import:**
 - Types-only: `import type { EvalSpec } from '@intentsolutions/core'` — pure compile-time; zero runtime weight.
 - Schemas: `import schema from '@intentsolutions/core/schemas/v1/gate-result.schema.json'` with `{ type: 'json' }` — JSON import; consumer can feed to `ajv / jsonschema-py / etc.`
 - Validators: `import { GateResultV1Schema } from '@intentsolutions/core/validators/v1/gate-result'` — loads `zod`; `GateResultV1Schema.parse(payload)` does the runtime check (`src/validators/v1/gate-result-v1.ts:42-72`).
 9. **Optional provenance verify:** `npm audit signatures @intentsolutions/core` — fetches the sigstore attestation and verifies. Failure point: signature missing / fails verification -> consumer must decide. The kernel itself doesn't enforce this; it's a consumer-side check.

Dependency Graph

Build-order dependencies (compile-time):

```

primitives.ts      (leaf - depends on nothing)
  ^
  |
state-machines/    (leaf - depends on nothing in src/)
  ^
  |
entities/          (depends on primitives, state-machines, predicates for FK typing)
  ^
  |
predicates/        (depends on primitives + state-machines; MUST NOT depend on entities)
  ^
  |
index.ts           (re-exports primitives + state-machines + entities + predicates)

```

Cross-cutting:

```

validators/v1/      (depends on `zod` from npm + sibling validator files;
                    MUST NOT depend on src/entities or src/predicates;
                    own copy of the predicate-body shape - twin source,
                    intentional, see "Tradeoff #3")

```

External:

```

@intentsolutions/audit-harness  (dev-only) - required for hash-pinning + arch + escape-scan
zod ^4.x                        (runtime, validators subpath only)

```

Failure modes if a dependency is unavailable:

- zod peer missing on consumer side -> consumer's bundler errors at import time of ./validators/v1/*. Types-only consumers unaffected.
- @intentsolutions/audit-harness unreachable during dev -> pnpm install fails locally, pnpm run harness:verify can't run, CI fails. Recovery: check npm registry; the harness is also on npm.
- TypeScript compiler unavailable -> pnpm run build fails; dist/ not regenerated. The previously-published tarball is unaffected because the published tarball already includes dist/.

Internal dep-cruiser-enforced rules (.dependency-cruiser.cjs:19-115):

Table 4.

Rule	Direction	Severity	Why
no-circular	any	error	Circular deps break tree-shaking and indicate layering rot
no-orphans	any	warn	Orphan modules are usually dead code

Rule	Direction	Severity	Why
no-deprecated-core	any -> Node deprecated core	warn	Avoid <code>punycode</code> , <code>domain</code> , etc.
kernel-no-runtime-deps	src/(?!validators/) -> npm	error	Kernel main entry must be zero-runtime-dep
validators-only-imports	src/validators/ -> npm except zod	error	Validators subtree is the only place that can import npm; zod is the only npm pkg it can import
predicates-no-entities	src/predicates -> src/entities	error	Predicates are the canonical signed surface; entities are the DB-side projection. Predicates must be definable without entities. EvidenceBundle imports from predicates (the reverse direction) — that's the canonical flow
no-test-imports-in-src	non-test -> *.test.ts	error	Test files cannot be imported from production source
state-machines-pure	src/state-machines -> src/(entities\\predicates)	error	state-machines is a leaf; entities depend on state-machines, never the reverse

That's the 8 forbidden architecture rules referenced throughout the docs.

4. Design Decisions & Tradeoffs

Decision Log

Decision #1 — Contracts-only boundary (no runtime in the kernel)

- **Chosen:** A strictly typed library with zero execution code, zero orchestration, zero LLM provider adapters, zero queues, and zero HTTP servers. Enforced architecturally via a 4-axis

boundary checker (`scripts/check-boundaries.ts`) reading `FORBIDDEN.md` + `ALLOWLIST.md`, plus eight dep-cruiser rules in `.dependency-cruiser.cjs`. The kernel never knows what an `EvalRun` does; it only knows the shape of one.

- **Over:** Bundling the kernel with a thin reference runtime (the “kernel + reference impl” pattern that some validators take), or shipping a CLI alongside the types, or including LLM provider adapters since “every consumer will need them anyway.”
- **Because:** Blueprint A § 3 anti-goals declare runtime execution, judges, harness logic, and services off-limits. Three concrete reasons this matters: (1) the kernel is what every downstream consumer pins; if it grows execution logic, every consumer inherits the dependency tree of every execution dep. (2) The kernel’s audience is polyglot — Python via Pydantic codegen (`iec-E08`), Rust if/when needed — and a TypeScript reference runtime would be useless to those consumers. (3) The unification thesis (DR-010 Q3 — every validator emits Evidence Bundle) only holds if the schema definition is decoupled from any particular validator implementation. A reference runtime would become the reference implementation by accident; a strict contracts-only kernel can’t.
- **Cost:** First-week engineers can’t run a “hello world” against the kernel directly. There is no example service, no demo, no `examples/` directory (in fact, `examples/` is on the `FORBIDDEN.md` Axis 3 list at line 280). Engineers have to read entity files + schemas + validators to understand the shapes; there is no executable demo. Some up-front cognitive load. Mitigation: the entity files are richly commented (every field has a JSDoc citing the Blueprint B section + line number), and the test fixtures in `tests/fixtures/v1/*.valid.json` serve as worked examples.
- **Revisit when:** A consumer needs a reference runtime AND nobody has written one in a sibling repo within a reasonable window. The expected answer is “write the reference runtime in a sibling repo, depend on the kernel.” If that pattern repeatedly breaks down, the boundary doctrine may need a Class-1 ISEDC convening to revisit.

Decision #2 — TypeScript-primary (with Python permitted for ML internals)

- **Chosen:** Author every canonical entity + predicate in TypeScript first, generate JSON Schema as the language-neutral form, and provide Zod runtime validators in TypeScript. Python distribution (Pydantic codegen) is deferred to `iec-E08` — not abandoned, just deferred.
- **Over:** Authoring the schemas in JSON Schema first and generating TS, OR authoring in Python (Pydantic) first and generating both TS and JSON Schema, OR maintaining all three forms hand-in-hand from day one with no primary.
- **Because:** DR-010 Q2 (ISEDC Session 4 widened-scope lock) binds the platform to “TS-primary signing surfaces; Python permitted ML internals.” The reasoning in DR-010 is: the signing surface — the in-toto + DSSE envelope path — is where supply-chain correctness lives. TS has the strongest mainstream tooling for compile-time type safety, the npm provenance flow has the strongest sigstore-keyless OIDC integration of any package ecosystem, and the platform’s overall consumer surface is JavaScript/TypeScript-heavy. Python remains permitted for the parts of the platform that are statistics or ML — but the canonical contracts must be authored in TS.
- **Cost:** Python consumers don’t have a Pydantic distribution at v0.1. They can consume `schemas/v1/*.schema.json` via `jsonschema-py`, but they don’t have brand-typed Pydantic models. That’s an actual gap. Also: the engineer authoring the kernel has to maintain three

forms in lockstep (TS + JSON Schema + Zod) rather than two — that’s the cost of forfeiting the JSON-Schema-first option.

- **Revisit when:** `iec-E08` (Pydantic codegen + Python distribution) ships. Until then, polyglot consumers fall back to JSON Schema only, and that’s a documented limit.

Decision #3 — JSON Schema + Zod twin sources (not one source, not codegen--canonical)

- **Chosen:** Maintain JSON Schema (`schemas/v1/*.schema.json`) and Zod validators (`src/validators/v1/*.ts`) as **twin sources**. Both are hand-authored. The codegen tool (`json-schema-to-zod` via `pnpm run codegen:validators`) produces a **reference output** at `src/validators/v1/_generated/` that is **not** the canonical Zod; the canonical Zod is hand-edited to add branded primitives, resolve cross-file `$refs`, use `z.discriminatedUnion` instead of `z.union`, and apply `superRefine` rules from `if/then` schema blocks.
- **Over:** (a) JSON-Schema-canonical with Zod auto-generated each build (single source). (b) Zod-canonical with JSON Schema auto-generated (single source via `zod-to-json-schema`). (c) Twin sources without any codegen reference.
- **Because:** Naive codegen in either direction loses information. JSON-Schema-to-Zod loses brands (no `.brand<'X'>()`), loses cross-file refs (becomes `z.any()`), loses `if/then` rules (becomes loose). Zod-to-JSON-Schema loses descriptions, custom error messages, and the ability to author the schema’s `$id` precisely (which matters for `gate-result/v1` because its `$id` is the canonical predicate URI). Twin sources without codegen reference would mean the two forms can drift silently. Twin-sources-with-codegen-as-reference (current choice) keeps both forms first-class, makes drift visible (when codegen output diverges meaningfully from canonical Zod, the engineer notices and reconciles), and locks the canonical Zod to the schemas via a battery of cross-validation tests (`src/__tests__/schemas.test.ts`, `src/validators/v1/validators.test.ts`, ERD-walk integration tests).
- **Cost:** Twice the surface to maintain. Every schema change requires a coordinated edit in both forms; the `_generated/` discipline (`src/validators/v1/_generated/README.md`) helps but doesn’t eliminate the manual reconciliation step. There is no automated “schemas + Zod are in sync” check beyond the fixture-driven cross-validation in tests. If a contributor edits one and not the other, the tests should catch it but the failure mode is “tests fail in a way that takes a minute to read” rather than “linter immediately flags drift.”
- **Revisit when:** Codegen tooling becomes sophisticated enough to round-trip brands + discriminated unions + `superRefine` rules without information loss. Or when the maintenance burden of twin-sourcing measurably slows the project down (which it has not yet, given the kernel ships 14 entity files + 14 schema files + 14 Zod validators + cross-validation tests). Class-2 ISEDC review would be the appropriate venue.

Decision #4 — `gate-result/v1` predicate body lives in the kernel as the only **NORMATIVE** predicate at `v1`

- **Chosen:** The `gate-result/v1` predicate body is **fully spec-bound at v1** — its full normative TS shape lives at `src/predicates/gate-result-v1.ts`, the full normative JSON Schema lives at `schemas/v1/gate-result.schema.json` with `$id` of `https://evals.intentsolutions.io/gate-result/v1.schema.json` (the predicate URI itself), and the Zod runtime validator at `src/validators/v1/gate-result-v1.ts` enforces the advisory-severity `if/then` rule via `superRefine`. Sibling predicate URIs

(`validation-result/v1`, `eval-verdict/v1`, `cost-attribution/v1`, `runtime-receipt/v1`) ship as **string-literal constants only** — their bodies are not yet spec-bound and they run in `sigstore_staging` per DR-010 Q3 conditional approval until each gets its own SPEC.md normative section.

- **Over:** (a) Wait to ship the kernel until all five predicate bodies are spec'd. (b) Ship none of them as normative; let consumers BYO predicate body. (c) Spec all five with looser shapes.
- **Because:** Blueprint B § 7.4 is the only fully-locked predicate body section at the Phase A foundation merge. The other four URIs are blessed as identifiers (Blueprint B § 3.3 / glossary § 6) but their bodies await their own SPEC.md normative sections. Shipping `gate-result/v1` immediately makes the unification thesis testable from day one — `audit-harness` can emit signed rows whose predicate body satisfies `GateResultV1Schema` right now; consumers don't have to wait for all five to be ready. The `sigstore_staging` vs `rekor_production` distinction (encoded in `EvidenceBundle.signing_mode` per Blueprint B § 2.4) keeps the unspec'd URIs out of production attestation flow.
- **Cost:** The kernel's `PREDICATE_URIS` constant (`src/predicates/gate-result-v1.ts:285-291`) exports five URIs but only one has a body type. Consumers who want to emit a `validation-result/v1` row have to BYO the body shape (which is fine — Blueprint B § 7.2 backward-compat policy allows adding URIs without a kernel version bump). There's some risk that a consumer mistakes the URI-as-identifier for "the body shape is locked" — the docstring in `src/predicates/gate-result-v1.ts:272-284` is explicit about this but humans skim docstrings. The CISO binding (predicate URIs MUST live at `evals.intentsolutions.io`, never `labs.intentsolutions.io`) is enforced by the boundary checker's URL-pattern axis (REFUSE — no override).
- **Revisit when:** Each sibling predicate URI gets its SPEC.md normative section ratified. At that point, the corresponding TS predicate body + JSON Schema + Zod validator land in the kernel under the same pattern as `gate-result/v1`. Deferral beads track each one.

Decision #5 — Sigstore-keyless provenance via npm OIDC, not signed releases with explicit keys

- **Chosen:** Use `pnpm publish --provenance` from the release workflow with `permissions: id-token: write`. The npm CLI exchanges the GitHub Actions OIDC token for a sigstore signing key (keyless / Fulcio-issued cert), signs the tarball, and posts the attestation to the registry. Consumers verify with `npm audit signatures @intentsolutions/core`.
- **Over:** (a) Maintaining a long-lived cosign key in repo secrets and signing with that. (b) GPG signatures on git tags only (no tarball signature). (c) No signing at all.
- **Because:** Keyless sigstore is the canonical npm provenance flow as of 2024+; it's tested, it's free, and the attestation lives at the registry next to the tarball so verification is one command. Long-lived keys add a key-rotation problem that we don't need. GPG-on-tags-only doesn't help consumers who install from npm because they're not verifying tags. No signing leaves the supply chain unsigned.
- **Cost:** Trust in the GitHub Actions OIDC issuer + sigstore's Fulcio CA + the npm registry. If any of those is compromised, the signature isn't reliable. Also: keyless sigstore certs are short-lived; the verification flow has to trust the transparency log (Rekor) for proof-of-existence. That's fine for npm packages but means the kernel can't trivially be air-gapped. Also: the consumer-side check (`npm audit signatures`) is opt-in; consumers who don't run it still get the unverified install. The kernel can't force consumer-side verification.

- **Revisit when:** Sigstore’s threat model changes substantially OR an npm package consumer needs offline / air-gapped verification OR the platform requires a stronger attestation framework (e.g., for FedRAMP-style compliance). Deferral beads **Security C-2** (Rekor pre-flight) and **Security C-3** (signing_mode in DSSE) track the broader DSSE story; the npm provenance flow is the v0.1 baseline.

Decision #6 — One published package (ONE BIG), not a monorepo of @intent-eval/types + @intent-eval/schemas + @intent-eval/validators

- **Chosen:** A single npm package @intentsolutions/core with subpath exports for each form: . for types, ./schemas/v1/*.json for JSON Schema, ./validators/v1/* for Zod (package.json:29-44). One package, one version, one tarball.
- **Over:** A pnpm workspace monorepo with separate @intentsolutions/eval-types, @intentsolutions/eval-schemas, @intentsolutions/eval-validators packages, each versioned independently.
- **Because:** DR-010 Q1 binds the platform to “ONE BIG” — a single canonical package, not a galaxy of smaller ones. Three reasons: (1) The three forms must be version-locked. If eval-types@1.2.0 declares EvalRun.id: Uuidv7 but eval-schemas@1.1.0 still has the old shape, consumers see drift in production. ONE BIG eliminates the multi-version-coordination problem by construction. (2) Subpath exports + tree-shaking deliver the same “pay for what you import” property that separate packages would: types-only consumers don’t load Zod (zero runtime cost via the main entry); validators consumers pull zod only when they import ./validators/v1/*. (3) Mental model: one consumer command (pnpm add @intentsolutions/core) covers every form. Onboarding is shorter.
- **Cost:** When the JSON Schema needs a fix but the TS types don’t (rare but possible), the consumer pulls a new version of all three forms. Bundle-cost-wise this is fine because of tree-shaking, but cognitive-cost-wise it can confuse someone who reads “v0.1.1 patch release” and wonders why they need to bump for a JSON-Schema-only fix. Also: the api-extractor SemVer gate (iec-E07) treats the public surface as one — a Zod validator change is a surface change even if the schemas didn’t move. That’s the right behavior but can occasionally feel coupled.
- **Revisit when:** A consumer subset emerges that legitimately needs only one form AND the bundle cost of ONE BIG materially exceeds the alternative AND the version-coordination problem of multi-package has been independently solved. Until any of those conditions land, ONE BIG holds.

Decision #7 — 100% coverage floor (not 80%, not “best effort”)

- **Chosen:** Set coverage.line / coverage.branch / coverage.function / coverage.statement to 100 in tests/TESTING.md thresholds, enforce in vitest.config.ts, and waive mutation testing (kill rate = 0). Excluded from coverage: src/validators/v1/_generated/ (codegen reference output, not consumer-shipped).
- **Over:** The conventional 80% floor, or “best effort” with no hard threshold.
- **Because:** This is a pure-type library. The “runtime code” is state-machine transition maps (literal-typed arrays), brand-discrimination helpers (canTransition), and Zod parsers that delegate to the framework. 100% coverage is the realistic and observed bar — there is no algorithmic complexity to flow through. If coverage drops below 100%, the cause is almost always “added a new branch and forgot to test it,” not “exotic uncoverable path.” Mutation testing is waived because mutations on as const arrays don’t add signal — mutating 'draft'

-> 'published' to 'draft' -> 'X' produces a TypeScript error, not a runtime mutation survivor.

- **Cost:** Adding a new entity that has a single new helper function requires writing tests for it before merge. Slows down “experimental sketch” PRs. But the kernel isn’t supposed to be a place for experimental sketches — it’s the contracts everyone depends on. The friction is by design.
- **Revisit when:** The kernel grows runtime logic with algorithmic complexity (it shouldn’t — that’s an anti-goal). At that point, mutation testing might be re-enabled and the coverage floor might be tuned down. Until then, 100% holds.

Decision #8 — Hash-pin the architecture policy (`.dependency-cruiser.cjs` + `FORBIDDEN.md` + `ALLOWLIST.md`)

- **Chosen:** Run `pnpm exec audit-harness init` after any edit to a policy file; it writes a SHA-256 digest to `.harness-hash`. The pre-commit hook + CI both run `pnpm run harness:verify`, which refuses to proceed if the current hash doesn’t match the pinned hash. Editing the policy without re-pinning fails the gate.
- **Over:** Trusting code review to catch policy edits, or relying on dep-cruiser alone (which doesn’t tamper-evident its own config).
- **Because:** Architecture rules are exactly the kind of thing an LLM contributor will quietly soften when stuck (“just disable that one rule”). Hash-pinning makes the softening *visible*: the contributor has to also commit the updated hash, which surfaces the intent in the diff. Code review can then evaluate “we are softening the no-runtime-deps rule because of X” rather than “this PR has a small dep-cruiser tweak buried in it.” This pattern is the IS Testing SOP standard (see umbrella 000-projects/CLAUDE.md § Intent Solutions Testing SOP).
- **Cost:** Adds a step to every legitimate policy edit. Engineer has to remember `pnpm exec audit-harness init` after the edit; if they forget, the pre-commit gate catches it but the iteration is one cycle longer. Documented in `ALLOWLIST.md` lines 100-117 + `FORBIDDEN.md` line 6.
- **Revisit when:** Never, probably. The cost is small; the value is “policy can’t drift silently.” This is the IS standard.

What Was Deliberately Not Built

Intentional omissions, in order of “most-likely-to-be-asked-about”:

- **No runtime / executor / orchestrator.** EvalRun execution does not live here. Forbidden by Blueprint A § 3 anti-goals; enforced by `FORBIDDEN.md` Axis 1 (no orchestration / job-queue / scheduler packages), Axis 2 (`src/runtime/`, `src/orchestrator/`, `src/scheduler/`, `src/queue/` all forbidden paths), and Axis 3 (`runtime/`, `orchestrator/`, `workers/`, `queues/` all forbidden top-level dirs). Owned by future runtime packages.
- **No LLM provider adapters.** No `openai`, `@anthropic-ai/*`, `@google-ai/*`, `langchain`, `llamaindex`, or anything else that calls a model API. Forbidden by Axis 1. Owned by `j-rig-skill-binary-eval` (behavioral evaluators) and `audit-harness` (deterministic gates).
- **No HTTP server / CLI / service surface.** No `express`, no `fastify`, no `bin` entry in `package.json`. The kernel is library-only. Forbidden by Axis 1 (web frameworks) and Axis 2 (`src/server/`, `src/api/`, `src/cli/`, `src/bin/`). Owned by future service packages if/when any exist.

- **No database / persistence.** No pg, mysql, mongodb, prisma, redis, etc. The kernel doesn't read or write any storage. Forbidden by Axis 1 + Axis 2 (src/db/, src/persistence/).
- **No Pydantic distribution at v0.1.** Polyglot consumers consume JSON Schema directly via jsonschema-py. Tracked at iec-E08 for future delivery; deferred per iec-E09 AAR's "What did NOT go in v0.1.0" section.
- **No automated migration codemod for downstream consumers.** The migration recipe lives in CHANGELOG.md § Adoption notes; consumers hand-migrate. Demoted to P2 per iec-E09 acceptance ("hand-migration acceptable").
- **No examples/ or demos/ directory.** Forbidden by Axis 3. The test fixtures (tests/fixtures/v1/*.valid.json) serve as worked examples; the entity files' JSDocs cite Blueprint B sections + line numbers.
- **No DSSE signature verification logic.** The kernel defines the DsseEnvelope + DsseSignature shapes (src/predicates/gate-result-v1.ts:249-269); cryptographic verification belongs in a sigstore client package downstream.
- **No AssertionExpression typed-class enum.** EvalSpec.assertions is typed as readonly unknown[]. Blueprint B § 2.1 says "typed assertion expression" without naming a class enum; the spec extraction explicitly directs **STOP — do not invent an assertion-class enum** (extension is a Class-2 ISEDC pair DR). Tracked at deferral bead bd_000-projects-gzgj (iec-deferral-A).
- **No ScoringConfig fields beyond aggregation_rule.** Blueprint B § 2.1 only spec-binds the aggregation rule; weights, thresholds, tiebreakers, confidence floors are intentionally absent — § 7.6 cordons threshold semantics off the predicate URI surface and into consumer-side tests/TESTING.md policy. Tracked at bd_000-projects-21re (iec-deferral-C).
- **No CompositionDag wire format normative spec.** The TS shape lives at src/entities/EvalSpec.ts:13 but the canonical serialization (adjacency vs edge list, ordering rules, etc.) isn't locked. Tracked at bd_000-projects-3sjx (iec-deferral-F).

Assumptions the Architecture Rests On

- **The npm registry is the durable distribution channel for at least the v0.x lifecycle.** If the registry goes away or its terms change materially, consumers lose the install path. No fallback is owned by this repo.
- **GitHub Actions OIDC + Fulcio + Rekor remain trustworthy.** The sigstore-keyless provenance flow trusts all three. If any is compromised, provenance attestations are unreliable.
- **Downstream sibling repos will migrate within a reasonable window.** The unification thesis (DR-010 Q3) only holds if consumers actually consume the kernel. Two weeks post-publish, the migration beads (iah-E02, iaj-E02, iel-link-schemas-to-kernel, intent-rollout-gate forthcoming) are still open. If they remain open indefinitely, the kernel's value proposition softens.
- **evals.intentsolutions.io will be DNSSEC + CAA pinned before first production-Rekor unlock.** CISO binding per DR-004 + DR-010 § 10. The predicate URI host is reserved; the cryptographic posture for the URI is a Security C-2 / C-3 deferral. Until then, signing_mode='rekor_production' is not a path consumers should take.
- **Blueprint B § 7.4 is stable enough that gate-result/v1 will not need a v2 in the v0.1 lifecycle.** Adding/loosening any field on the predicate body requires Class-1 ISEDC convening; that's deliberate friction. If Blueprint B § 7.4 needs revision more than once per quarter, the friction is wrong-sized.

- TypeScript’s branded-type pattern (`declare const __brand: unique symbol`) keeps working across compiler versions. It does today (TS 5.7); a future TS release that changes phantom-type semantics would force a migration. Mitigation: tsd negative tests would catch the regression at upgrade time.
- **Zod 4.x stays binary-compatible enough that consumers can pin a range.** ALLOWLIST.md pins zod ^4.4.3. A Zod 5 release would force a kernel bump.

5. Directory Structure

Layout

```
intent-eval-core/
+- .github/
| +- workflows/
|   +- ci.yml                # 9-step gate chain on push + PR
|   +- boundary-check.yml    # 4-axis + URL pattern check on push + PR
|   +- release.yml           # tag-triggered npm publish with --provenance
|
+- .husky/
| +- pre-commit              # escape-scan --staged; boundaries; lint-staged
|
+- 000-docs/                  # repo-local docs per Doc Filing Standard v4.3
| +- 000-INDEX.md
| +- 001-AA-AACR-release-v0.1.0-... # v0.1.0 release after-action report
| +- 002-AT-ARCH-repo-blueprint-... # NORMATIVE per-repo blueprint (iec-E10)
| +- 003-AT-STND-core-repo-boundaries # NORMATIVE 4-axis boundary doctrine (iec-E11)
| +- 004-AT-AAR-testing-cicd-...    # iec-E12 ratify-and-close AAR
| +- 005-AA-AUDT-appaudit-...       # this file
|
+- api/
| +- intentsolutions-core.api.md    # api-extractor golden snapshot (SemVer gate)
| +- temp/                          # api-extractor working dir (regenerable)
|
+- reports/
| +- arch/dep-cruiser.log           # dep-cruiser output capture
|
+- schemas/v1/                   # JSON Schema (draft 2020-12) wire format
| +- _common.schema.json           # 14 shared $defs (uuidv7, sha256Prefixed, etc.)
| +- index.json                    # catalog
| +- gate-result.schema.json        # NORMATIVE predicate body (Blueprint B § 7.4)
| +- eval-spec.schema.json
| +- eval-run.schema.json
| +- matcher-map.schema.json
| +- evidence-bundle.schema.json
| +- judge-decision.schema.json
| +- runtime-receipt.schema.json
| +- regression-pack.schema.json
```

```

| +- rollout-gate.schema.json
| +- skill-snapshot.schema.json
| +- session-trace.schema.json
| +- tool-invocation.schema.json
| +- cost-record.schema.json
| +- failure-taxonomy.schema.json
|
+- scripts/
| +- check-boundaries.ts          # 4-axis boundary checker
| +- api-diff.ts                  # api-extractor diff helper for CI
|
+- src/
| +- index.ts                     # public surface - re-exports
| +- index.test.ts                # smoke test
| +- primitives.ts                # branded primitive types (leaf)
| +- integration.test.ts          # ERD-walk integration test
| +- __tests__/
| | +- schemas.test.ts           # ajv against JSON Schemas + fixtures
| +- entities/                   # 13 canonical entity TS interfaces
| | +- index.ts                  # barrel re-export
| | +- EvalSpec.ts               # § 2.1
| | +- EvalRun.ts                # § 2.2
| | +- MatcherMap.ts             # § 2.3
| | +- EvidenceBundle.ts         # § 2.4
| | +- JudgeDecision.ts          # § 2.5
| | +- RuntimeReceipt.ts         # § 2.6
| | +- RegressionPack.ts         # § 2.7
| | +- RolloutGate.ts            # § 2.8
| | +- SkillSnapshot.ts          # § 2.9
| | +- SessionTrace.ts           # § 2.10
| | +- ToolInvocation.ts         # § 2.11
| | +- CostRecord.ts             # § 2.12
| | +- FailureTaxonomy.ts        # § 2.13
| | +- evidence-judge-receipt.test.ts # cluster test
| | +- regression-rollout-skill.test.ts # cluster test
| | +- session-tool-cost-failure.test.ts # cluster test
| +- predicates/
| | +- index.ts                  # barrel re-export
| | +- gate-result-v1.ts         # NORMATIVE predicate body (Blueprint B § 7.4)
| | +- gate-result-v1.test.ts
| +- state-machines/
| | +- types.ts                  # TransitionMap<S> + canTransition (leaf)
| +- validators/v1/
| | +- index.ts                  # barrel re-export
| | +- _primitives.ts            # Zod branded primitives ($defs counterparts)
| | +- gate-result-v1.ts         # GateResultV1Schema with superRefine
| | +- eval-spec.ts              # one per entity
| | +- eval-run.ts

```

```

|     +- (... 13 more entity validators ...)
|     +- validators.test.ts           # round-trip fixture tests
|     +- _generated/                  # codegen reference output (NOT canonical)
|     +- README.md                    # discipline doc
|     +- (... one .ts per schema ...)
|
+- test-d/                            # tsd negative type tests against dist/
| +- primitives.test-d.ts
| +- predicates.test-d.ts
| +- state-machines.test-d.ts
|
+- tests/
| +- TESTING.md                      # engineer-owned policy (hash-pinned)
| +- RTM.md                          # 24 REQ-IEC-NNN requirements traceability
| +- PERSONAS.md                     # 4 downstream-consumer personas
| +- JOURNEYS.md                     # 3 type-level journeys
| +- fixtures/v1/                    # positive + negative golden fixtures (JSON)
|   +- gate-result.valid.json
|   +- gate-result.invalid-bad-decision.json
|   +- gate-result.invalid-bad-hash-format.json
|   +- gate-result.invalid-missing-gate_name.json
|   +- gate-result.advisory.valid.json
|   +- eval-spec.valid.json
|   +- eval-spec.invalid-bad-aggregation.json
|   +- (... 14 more positive fixtures for the other entities ...)
|
+- (top-level governance files)
| +- README.md
| +- CHANGELOG.md
| +- CLAUDE.md                       # Claude-Code-specific guidance
| +- AGENTS.md                       # vendor-neutral cross-CLI agent contract
| +- LICENSE                          # Apache 2.0
| +- CODEOWNERS                      # PR review routing
| +- FORBIDDEN.md                    # 4-axis boundary forbidden set
| +- ALLOWLIST.md                    # runtime + devDep + top-level files allow set
| +- TEST_AUDIT.md                   # /audit-tests output (iec-E12c)
| +- .harness-hash                   # hash-pin manifest for policy files
| +- .dependency-cruiser.cjs          # 8 forbidden arch rules
| +- api-extractor.json
| +- package.json
| +- pnpm-lock.yaml
| +- tsconfig.json                   # noEmit, for editor + lint
| +- tsconfig.build.json              # emit-only build config
| +- vitest.config.ts
| +- eslint.config.js
| +- .prettierrc.json
| +- .prettierignore
| +- .gitignore

```

Load-Bearing Files

These five-to-ten files are what hold the kernel up. If any of them breaks, something material breaks.

1. **src/predicates/gate-result-v1.ts** — The full NORMATIVE TS shape for the `gate-result/v1` predicate body. Holds the `SUBJECT_NAME_REGEX`, `GATE_RESULT_V1_URI` const, all four closed enums (`GateDecision`, `AdvisorySeverity`, `ReplayFidelityLevel`, `SubjectSide`), the `GateResultV1Required` + `GateResultV1Optional` interfaces, and the in-toto Statement / DSSE envelope types. If this drifts from Blueprint B § 7.4, every downstream signed attestation is wrong-shaped.
2. **schemas/v1/gate-result.schema.json** — The JSON Schema counterpart of (1), with `$id` of `https://evals.intentsolutions.io/gate-result/v1.schema.json` (the URI literally is the schema id). Consumed by polyglot consumers (Python via `jsonschema-py`, etc.). Same constraint: must mirror the TS shape.
3. **src/validators/v1/gate-result-v1.ts** — The Zod runtime validator counterpart of (1) and (2). Owns the `superRefine` that enforces the advisory-severity if/then rule. Twin-source with the JSON Schema (see Decision #3).
4. **src/primitives.ts** — The 10 branded primitive types. Every entity FK and every hash field across the kernel imports from here. If brand naming changes, every consumer's branded-string handling breaks.
5. **FORBIDDEN.md** + **ALLOWLIST.md** + **scripts/check-boundaries.ts** — The 4-axis boundary doctrine. If any goes silent (e.g., a parser regression silently classifies as 0-violations on an actually-violating change), runtime code can slip into the kernel.
6. **.dependency-cruiser.cjs** — The 8 architecture rules. Same threat model: if a rule is silently weakened, arch invariants weaken.
7. **.harness-hash** — The hash-pin manifest. If this is stale (i.e., doesn't reflect the current `.dependency-cruiser.cjs` content), the harness verify step starts blocking commits — which is correct behavior; the file failing safe is by design. The risk is the *opposite*: if the file is regenerated without scrutiny on a policy weakening, the audit trail breaks.
8. **.github/workflows/release.yml** — The publish flow. Holds the tag-vs-package check, the full gate chain re-run, and the `pnpm publish --provenance` step. If this regresses, releases can publish unsigned or wrong-versioned.
9. **package.json#exports** — Defines the public subpath surface (`.`, `./schemas/v1/*.json`, `./schemas/v1`, `./validators/v1`, `./validators/v1/*`). If this drops a subpath, consumer imports break in the next published version.
10. **api/intentsolutions-core.api.md** — The api-extractor golden snapshot. The SemVer regression gate diffs against this. If it's committed wrong (or not committed at all), the surface-stability promise is silently broken.

6. Getting Started

Prerequisites

Table 5.

Tool	Version	Install	Verify
Node.js	>=20 (Node 22 pinned for dev via <code>.nvmrc</code>)	<code>https://nodejs.org</code> or <code>fnm install 22 && fnm use 22</code>	<code>node --version -> v22.x.x</code>
pnpm	>=9 (pinned 9.15.0 via <code>packageManager</code> in <code>package.json</code>)	<code>corepack enable && corepack prepare pnpm@9.15.0 --activate</code>	<code>pnpm --version -> 9.15.0</code>
git	any modern	distro package manager	<code>git --version</code>
node --experimental-strip-types support	comes with Node 22	n/a	<code>node --experimental-strip-types -e 'console.log(1)' -> 1</code>

Zero to Running

The kernel is a library, not a service, so “running” means “tests pass and you can import a type.”

1. **Clone:**

```
git clone git@github.com:jeremylongshore/intent-eval-core.git
cd intent-eval-core
```

2. **Install** (deterministic via lockfile):

```
pnpm install --frozen-lockfile
```

Expect: pnpm fetches deps + sets up husky hooks via the `prepare` script (`package.json:57`). Output ends with “Done in Xs.”

3. **Run the canonical gate** (this is `pnpm run check` per `package.json:76`):

```
pnpm run check
```

Steps it runs in order: `lint -> typecheck -> test` (vitest run) `-> arch` (audit-harness arch) `-> boundaries` (4-axis check). All green is the expected outcome on a clean checkout of `main`. Coverage and dist artifact verification are CI-only — they are not part of `check` but are part of `ci.yml`.

4. Import a type to confirm:

```
pnpm exec node --input-type=module -e "
  import('./dist/index.js').then(m => console.log(Object.keys(m).slice(0, 10)));
"
```

First run `pnpm run build` if `dist/` doesn't exist yet. Expect a list starting with state-machine transition maps + the `canTransition` helper + the `GATE_RESULT_V1_URI` const. (Type-only exports don't appear in JS object keys at runtime — that's fine; types are erased.)

5. Validate a sample bundle against `gate-result/v1` (the load-bearing demo):

The repo doesn't have a stand-alone “run me” example by design (Axis 3 forbids `examples/`), but the test fixtures + validator give you what you need. From a Node REPL after build:

```
const { GateResultV1Schema } = await import('./dist/validators/v1/gate-result-v1.js');
const valid = (await import('node:fs/promises'))
  .readFile('tests/fixtures/v1/gate-result.valid.json', 'utf-8')
  .then(JSON.parse);
GateResultV1Schema.parse(await valid); // returns the parsed + branded object
```

To see the failure mode, swap the fixture path for one of the `invalid-*.json` fixtures. The `superRefine` advisory rule fires for the `gate-result.invalid-bad-decision.json` case; the strict-object rejection fires for the `missing-gate_name` case.

6. Understand the `gate-result/v1` predicate:

Read in order: `src/predicates/gate-result-v1.ts` -> `schemas/v1/gate-result.schema.json` -> `src/validators/v1/gate-result-v1.ts` -> `tests/fixtures/v1/gate-result.valid.json`. Each form is the same shape from a different angle. The TS file has the richest JSDoc citing Blueprint B § 7.4 line numbers (the docstrings are part of the load-bearing surface — they explain why each field exists). The JSON Schema is the wire form. The Zod validator is what runs at consumer runtime.

Common Setup Problems

Table 6.

Symptom	Cause	Fix
<code>pnpm install</code> fails with <code>ERR_PNPM_UNSUPPORTED_ENGINE</code>	Node version too old (<20)	Install Node 22 via <code>nvm/fnm/corepack</code> ; the <code>.nvmrc</code> is the source of truth
<code>pnpm install</code> fails with <code>Cannot find module 'husky' (chicken-and-egg)</code>	The <code>prepare</code> script runs <code>husky</code> before <code>husky</code> is installed in some edge cases	Run <code>pnpm install --ignore-scripts</code> first, then <code>pnpm rebuild husky</code>

Symptom	Cause	Fix
<code>pnpm run boundaries</code> fails with <code>cannot find module 'check-boundaries.ts'</code>	Older Node (<22.6) doesn't support <code>--experimental-strip-types</code>	Upgrade to Node 22 per <code>.nvmrc</code> ; the script's shebang <code>#!/usr/bin/env -S node</code> <code>--experimental-strip-types</code> requires it
Pre-commit refuses with <code>harness-hash</code> mismatch	You edited <code>.dependency-cruiser.cjs</code> , <code>FORBIDDEN.md</code> , or <code>ALLOWLIST.md</code> without re-pinning	Run <code>pnpm exec audit-harness init</code> ; commit the updated <code>.harness-hash</code> in the same commit
<code>pnpm run check</code> fails on <code>arch</code> step with <code>kernel-no-runtime-deps</code>	You imported a third-party npm package from somewhere in <code>src/</code> outside <code>src/validators/</code>	Either move the dep to a sibling repo (correct fix) or — if you genuinely need a new runtime dep — file an <code>iec-boundary-override</code> bead and update <code>ALLOWLIST.md</code> 's table
<code>pnpm run check</code> fails on <code>boundaries</code> step with <code>REFUSE</code> on <code>labs.intentsolutions.io</code>	You referenced <code>labs.intentsolutions.io</code> as a predicate URI host in <code>code/schema/fixture</code>	This is CISO-binding-forbidden per DR-004 + DR-010 § 10. There is no override. Use <code>evals.intentsolutions.io</code> .
<code>pnpm publish</code> fails with <code>EOTPNEEDED</code>	You're publishing locally instead of from CI	Don't publish locally. Push a tag (<code>git tag -a vX.Y.Z; git push origin vX.Y.Z</code>); the release workflow publishes from CI with provenance
<code>npm audit signatures</code> <code>@intentsolutions/core</code> returns “no signatures found”	Consumer's npm version too old (<9.5)	Upgrade npm: <code>npm install -g npm@latest</code> ; the provenance attestation requires npm CLI's <code>audit-signatures</code> command

Symptom	Cause	Fix
<pre>import schema from '@intentsolutions/core/schemas/v1/gatesolver.json' or with { type: 'json' } errors in TS</pre>	Project lacks <code>tsconfig.json</code> or the with syntax requires <code>module: "NodeNext"</code>	Set <code>"resolveJsonModule":</code> <code>true + "module":</code> <code>"NodeNext"</code> (or <code>"NodeNext"-</code> compatible) in consumer <code>tsconfig.json</code>

7. Operations

Command Map

Table 7.

Task	Command	Notes
Install deps	<code>pnpm install --frozen-lockfile</code>	Frozen lockfile mode is what CI uses; matches dev to CI
Run all checks (the canonical pre-commit gate)	<code>pnpm run check</code>	lint + typecheck + test + arch + boundaries (5 steps; cf. <code>package.json:76</code>)
Run unit tests	<code>pnpm run test</code>	vitest run mode
Run tests in watch mode	<code>pnpm run test:watch</code>	vitest in watch
Run tests with coverage	<code>pnpm run test:coverage</code>	Enforces the 100% floor via <code>vitest.config.ts</code>
Run negative type tests	<code>pnpm run test:types</code>	tsd against <code>test-d/</code> files; second-opinion against published surface
Lint	<code>pnpm run lint</code>	eslint flat config
Lint and fix	<code>pnpm run lint:fix</code>	
Typecheck only	<code>pnpm run typecheck</code>	<code>tsc --noEmit</code>
Format	<code>pnpm run format</code>	<code>prettier --write .</code>
Format check (CI mode)	<code>pnpm run format:check</code>	<code>prettier --check .</code>
Build	<code>pnpm run build</code>	<code>tsc -p tsconfig.build.json -> dist/</code>

Task	Command	Notes
Architecture rules	<code>pnpm run arch</code>	audit-harness arch -> dep-cruiser with 8 forbidden rules
4-axis boundary check	<code>pnpm run boundaries</code>	scripts/check-boundaries.ts
Regenerate Zod codegen reference	<code>pnpm run codegen:validators</code>	Writes to src/validators/v1/__generated/; NOT canonical
Verify hash-pinned policy artifacts	<code>pnpm run harness:verify</code>	Refuses if <code>.harness-hash</code> doesn't match current policy file digests
Re-pin hash-pinned policy	<code>pnpm exec audit-harness init</code>	After legitimate edits to <code>.dependency-cruiser.cjs</code> / <code>FORBIDDEN.md</code> / <code>ALLOWLIST.md</code>
Extract API surface snapshot	<code>pnpm run api:extract</code>	Build + api-extractor; refreshes <code>api/intentsolutions-core.api.md</code>
Check API surface against snapshot	<code>pnpm run api:check</code>	Same as above but errors on drift
Print API diff vs snapshot	<code>pnpm run api:diff</code>	Helper output for CI when surface drifts
View logs	<code>gh run view <run-id> (Actions)</code>	No own-service logs; CI logs in Actions UI
Deploy "staging"	n/a	No staging registry; tag goes straight to npm
Deploy production (publish to npm)	<code>git tag -a vX.Y.Z -m "Release vX.Y.Z" && git push origin vX.Y.Z</code>	release.yml fires on tag; CI publishes
Rollback	<code>npm deprecate '@intentsolutions/core@X.Y.Z' '<reason>'</code>	npm packages can't be unpublished after 72h; deprecation is the recovery path

Deployment

This is a library that publishes to the public npm registry. The "deployment" is the release workflow.

Pre-flight checklist (engineer running the release):

- ☐ Working tree clean (`git status` is clean).

- ☐ `main` is up to date with `origin/main` (`git fetch origin && git diff origin/main -> empty`).
- ☐ CI on `main` is green (last commit's CI run is success).
- ☐ `pnpm run check` passes locally.
- ☐ `pnpm run test:coverage` passes locally (or trust CI).
- ☐ `pnpm run build && pnpm run test:types` passes (api-extractor surface is consistent with the dist build).
- ☐ `package.json#version` has been bumped to the target X.Y.Z per SemVer.
- ☐ `CHANGELOG.md` has an entry for X.Y.Z under the Keep-a-Changelog format.
- ☐ `api/intentsolutions-core.api.md` reflects the current surface (run `pnpm run api:extract` and commit if anything changed).
- ☐ Release notes drafted (the GH release page reuses the `CHANGELOG` entry).

Execution steps:

1. Tag

```
git tag -a v0.1.1 -m "Release v0.1.1"
```

2. Push the tag (this fires `release.yml`)

```
git push origin v0.1.1
```

The release workflow then:

1. Checks out with full history.
2. Sets up pnpm + Node + registry URL.
3. `pnpm install --frozen-lockfile`.
4. Verifies tag matches `package.json` version (the canonical drift defense).
5. `pnpm run harness:verify` (the hash-pin gate).
6. `pnpm run lint, typecheck, arch, test, test:coverage, build, test:types`.
7. Verifies dist artifacts emitted (`dist/index.js`, `dist/index.d.ts`, `dist/validators/v1/index.js`, `schemas/v1/gate-result.schema.json`).
8. `pnpm publish --no-git-checks --provenance --access public` with `NODE_AUTH_TOKEN` from secrets + `id-token: write` for sigstore.

Verification:

- npm registry shows the new version (typically within seconds of publish step success): `npm view @intentsolutions/core versions`.
- Provenance attestation present: `npm audit signatures @intentsolutions/core` returns 1 signature on the dist tarball.
- GH release page created (the AAR workflow + manual `gh release create` step — the v0.1.0 AAR documented this path).

Rollback protocol:

1. If the bad version has been live <72h AND no consumers have pinned it,

you MAY unpublish - but treat as last resort:

```
npm unpublish @intentsolutions/core@X.Y.Z
```

2. Otherwise (standard path), deprecate:

```
npm deprecate '@intentsolutions/core@X.Y.Z' \
'Deprecated due to <reason>; use X.Y.Z+1 instead'
```

```
# 3. Cut a fix release as X.Y.Z+1:
#   - fix the issue on main
#   - bump package.json#version to X.Y.Z+1
#   - update CHANGELOG.md
#   - commit + push to main (wait for CI green)
#   - tag + push
git tag -a vX.Y.Z+1 -m "Release vX.Y.Z+1 - fix for <reason>"
git push origin vX.Y.Z+1
# (release.yml fires; npm 'latest' tag auto-points to vX.Y.Z+1)
```

Monitoring & Alerting

Table 8.

Surface	Status
Dashboards	Not configured. The npm registry shows download counts, dependent count, last publish time; no Intent-Solutions-owned dashboards.
SLIs / SLOs	Not defined. The library is library-only — there’s no service to define availability against. The closest analog is “every consumer who installs a pinned version always gets a green install + a verifiable signature.”
On-call rotation	Not established. The single author + Anthropic-cohort backup is the rotation.
Alerting	Consumer-side via npm advisory feed if a CVE is filed against the package. Otherwise none.

Incident Response

Table 9.

Severity	Definition	Response Time	Playbook
P0	Published version is malicious (e.g., wrong tarball uploaded, account compromise)	Immediate	Within 72h: npm unpublish . After 72h: npm deprecate + cut a new fix release. Rotate npm token. Notify consumers via GH release notes + (forthcoming) Discord.
P1	Published version has a critical correctness bug (e.g., GateResultV1Schema rejects valid payloads)	Immediate	Deprecate the bad version. Fix on main. Tag + publish a patch release. Update CHANGELOG with the migration note.
P2	Sigstore / npm provenance attestation fails for the published tarball	1 hour	Investigate Actions OIDC token issuance; cut a new patch release that re-runs the provenance flow. Notify consumers verifying via npm audit signatures .
P3	API surface drift slipped through (e.g., minor bump that broke consumers)	1 day	Cut a patch release that restores the broken surface (the api-extractor gate at iec-E07 should prevent this from happening but is not impossible to bypass).

8. Things That Will Bite You

Ordered by likelihood x impact.

8.1 JSON Schema and Zod validator drift (the twin-source maintenance burden)

- **Symptom:** A schema change lands in `schemas/v1/<entity>.schema.json` but the corresponding `src/validators/v1/<entity>.ts` is missed. The CI test suite catches it via fixture cross-validation, but the failure message is “vitest test failed: parse threw” rather than “schemas and validators are out of sync.” Engineer scratches head.
- **Cause:** The twin sources are hand-authored (Decision #3). Codegen produces only the `_generated/` reference output. There is no single check that says “these two are in sync.”
- **Fix:** When the test fails, read the test name in the failure output — the cross-validation test in `src/__tests__/schemas.test.ts` + `src/validators/v1/validators.test.ts` names the entity. Open both the schema file and the validator file side-by-side and reconcile.
- **Prevention:** Always edit both files in the same PR. Use the codegen reference (`pnpm run codegen:validators`) as a sanity check on the structural shape before propagating brands + `superRefine` rules to the canonical Zod. Engineer convention only; not enforced.

8.2 Forgetting to regenerate `_generated/` after a schema change

- **Symptom:** `_generated/` reflects an old schema version. It’s not the canonical Zod, so consumer correctness is unaffected, but engineers reading `_generated/` for a “structural shape sanity check” get stale info and may carry old assumptions forward.
- **Cause:** `_generated/` is the codegen output of `pnpm run codegen:validators`. The script is reproducible but it’s manual — there’s no pre-commit hook that re-runs codegen.
- **Fix:** `pnpm run codegen:validators` and commit the updated `_generated/` files alongside the schema change. Or accept that `_generated/` is stale and ignore it for that PR.
- **Prevention:** Document at `src/validators/v1/_generated/README.md` says to consult `_generated/` “before hand-authoring a new entity validator” and “when updating an existing schema.” Engineer convention only.

8.3 Hash-pin mismatch after a policy edit (the harness-hash gate)

- **Symptom:** Pre-commit refuses with “harness-hash mismatch” or CI’s `harness:verify` step fails red.
- **Cause:** You edited `.dependency-cruiser.cjs`, `FORBIDDEN.md`, or `ALLOWLIST.md` (these are hash-pinned per `ALLOWLIST.md` lines 100-117) without running `pnpm exec audit-harness init` to refresh `.harness-hash`.
- **Fix:** Run `pnpm exec audit-harness init`. Stage `.harness-hash`. Commit alongside the policy edit (same commit ideal; separate is OK).
- **Prevention:** This is the IS standard — there is no way around it without bypassing the gate. Don’t try to bypass; it’s there for a reason (Decision #8).

8.4 The 8 forbidden architecture rules — knowing which one fired and what to do

- **Symptom:** `pnpm run arch` fails red with a dep-cruiser violation. The 8 rules and their fix paths (cf. `.dependency-cruiser.cjs:19-115`):

Table 10.

Rule	If it fires it means	Fix
<code>no-circular</code>	You added an import that creates a cycle between modules	Refactor — circular deps are an error, not a warning
<code>no-orphans</code> (warn)	You added a module nothing imports	Either import it from somewhere legitimate or delete
<code>no-deprecated-core</code> (warn)	You imported <code>punycode</code> / <code>domain</code> / etc.	Use a modern alternative or polyfill
<code>kernel-no-runtime-deps</code>	You imported an npm package from somewhere outside <code>src/validators/</code>	Move it to <code>src/validators/</code> (if it's zod) or to a sibling repo (everything else)
<code>validators-only-import-zod</code>	You imported an npm package other than zod from <code>src/validators/</code>	Move the dep elsewhere; the validators subtree is zod-only by design
<code>predicates-no-entities</code>	You imported from <code>src/entities/</code> in <code>src/predicates/</code>	Invert the dependency — entities can import predicates (canonical direction); predicates cannot import entities
<code>no-test-imports-in-src</code>	You imported a <code>*.test.ts</code> file from production source	Tests are not consumer-shipped; move the imported helper to a non-test file
<code>state-machines-pure</code>	You imported from <code>src/entities/</code> or <code>src/predicates/</code> in <code>src/state-machines/</code>	<code>state-machines</code> is a leaf layer — invert

- **Cause:** Each rule encodes a design invariant; the rule message includes the rationale.
- **Fix:** Read the rule's `comment` field in `.dependency-cruiser.cjs` — the inline rationale is explicit. Fix the import; do NOT add an exclusion.
- **Prevention:** When introducing a new module under `src/`, mentally walk through the eight rules first. “Does it import an npm package? Does it sit in validators? Does it cross the predicates/entities boundary?”

8.5 Forbidden URL pattern violation (`labs.intentsolutions.io`)

- **Symptom:** `pnpm run boundaries` fails with severity `REFUSE` on a `labs.intentsolutions.io` URL match.

- **Cause:** You wrote `labs.intentsolutions.io` as a predicate URI host somewhere in source, schema, fixture, or test. The CISO binding (DR-004 + DR-010 § 10) forbids this — predicate URIs MUST live at `evals.intentsolutions.io`; `labs.intentsolutions.io` may host blog/methodology content but never an in-toto predicate URI, OTEL attribute namespace, or attestation predicate identifier.
- **Fix:** Change the URL to `evals.intentsolutions.io`. **There is no override.** The boundary checker emits exit code 2 (REFUSE), not 1 (CHALLENGE). PR descriptions cannot soften this.
- **Prevention:** Always use `evals.intentsolutions.io` for any predicate URI / attestation identifier. The FORBIDDEN.md doctrine has an explicit “does this line document the rule?” exemption (so this file and the doctrine file can mention the forbidden URL without being flagged), but the checker is conservative.

8.6 npm provenance verification failure on consumer side

- **Symptom:** Consumer runs `npm audit signatures @intentsolutions/core` and gets “verification failed” or “no signatures found.”
- **Cause** (several possible): (a) Consumer’s npm CLI version is too old (<9.5). (b) Sigstore Fulcio CA chain is unreachable in the consumer’s network. (c) The published version was *actually* published without `--provenance` (which should never happen given the release workflow, but is possible if someone manually published).
- **Fix:** First, upgrade npm: `npm install -g npm@latest`. Second, check network reachability to `fulcio.sigstore.dev`. Third, verify the version was indeed published from the release workflow (`gh run list --workflow=release.yml --branch=<tag>`).
- **Prevention:** The release workflow always passes `--provenance`. Manual local publishing is the only way to bypass it, and shouldn’t happen. If the publish-from-laptop habit creeps in, file a bead to forbid it explicitly.

8.7 Tag-vs-package version mismatch on release

- **Symptom:** Release workflow fails with `::error::Tag vX.Y.Z does not match package.json version A.B.C.`
- **Cause:** You tagged before bumping `package.json#version`, or you tagged the wrong version.
- **Fix:** Delete the bad tag (`git tag -d vX.Y.Z`; `git push origin :refs/tags/vX.Y.Z`), bump `package.json#version` correctly, commit, push, re-tag, push tag. Note: deleting a remote tag is allowed pre-release; do not do it post-publish.
- **Prevention:** The release workflow’s tag-vs-package check (`.github/workflows/release.yml:52-60`) is what catches this — that’s the right gate. Just don’t push the bad tag at all if you can help it.

8.8 Adding a 9th runtime dep without ALLOWLIST.md update

- **Symptom:** `pnpm run boundaries` fails with “ALLOWLIST cap exceeded” or a missing-allowlist-entry error.
- **Cause:** You added a runtime dep to `package.json#dependencies` without first updating `ALLOWLIST.md` (cap of ≤ 8 at v0.1) and re-pinning the harness hash.
- **Fix:** Either move the dep to `devDeps` (if dev-only), or file an `iec-` boundary-override bead, update `ALLOWLIST.md`’s table, run `pnpm exec audit-harness init`, and reference the bead in the PR body (`boundary-override: bd_000-projects-<id>`).

- **Prevention:** Before adding any runtime dep, read ALLOWLIST.md’s table and check current count. If you’re adding the second runtime dep (current count is 1), confirm it’s justified.

8.9 Forgetting to refresh the api-extractor golden snapshot

- **Symptom:** CI fails on `api:check` with “Public API surface has drifted from the committed golden snapshot.”
- **Cause:** You added or changed a public export (entity, validator, primitive, predicate type) without running `pnpm run api:extract` to update `api/intentsolutions-core.api.md`.
- **Fix:** Run `pnpm run api:extract`, review the diff, commit the updated snapshot. If the change is a breaking change, also bump the major version per per-repo blueprint § 11.1.
- **Prevention:** When touching any file in `src/index.ts`’s transitive surface, plan to refresh the snapshot in the same PR.

8.10 Consumer imports the codegen reference (`_generated/`) by accident

- **Symptom:** Consumer’s bundle is larger than expected, or consumer’s types don’t have the expected brands.
- **Cause:** Consumer wrote `import { Foo } from '@intentsolutions/core/validators/v1/_generated/...` — which works because pnpm’s resolution doesn’t strictly enforce the `package.json#exports` map for deep paths.
- **Fix:** Consumer changes the import path to `@intentsolutions/core/validators/v1/foo` (without the `_generated/`).
- **Prevention:** `src/validators/v1/_generated/README.md` is explicit (“Don’t import from `_generated/`”) but readers may skip it. The `package.json exports` field does NOT export `_generated/*` (cf. `package.json:36-43`), so deep imports via the export map are blocked; only manual deep-path imports through `node_modules` bypass it. The risk is small but real.

9. Security & Access

Access Control

Table 11.

Role	Purpose	Permissions	MFA
@jeremylongshore (owner)	Solo maintainer	Admin on GH repo + npm package	Yes (per repo + npm requirements)
GitHub Actions runner	CI + release workflow	Read repo contents, write id-token (OIDC) for sigstore, write packages via <code>NODE_AUTH_TOKEN</code>	Workflow scope; secrets gated by branch protection
Codeowner routing	PR review	@jeremylongshore per CODEOWNERS	n/a

Role	Purpose	Permissions	MFA

Secrets

- **Where:** `NPM_TOKEN` lives in GitHub Actions repo secrets. The `id-token: write` permission is granted per-workflow (`release.yml`) and exchanged for a sigstore signing cert via the npm provenance flow. No long-lived signing keys are stored anywhere in the repo or in CI secrets — sigstore-keyless OIDC is the whole point.
- **Rotation:** The `NPM_TOKEN` is the only long-lived secret; rotate it via npm account UI when needed (e.g., suspected compromise, periodic cycle). Update the GH Actions secret in the same step. No automation.
- **Emergency access:** If the npm token is compromised, immediate rotation + (within 72h) `npm unpublish` the affected versions if needed, then re-tag and re-publish from a clean rotation. Sigstore provenance from the new token + the same GH OIDC issuer makes the provenance chain re-verifiable end-to-end.

Honest Security Assessment

Implemented:

- Sigstore-keyless provenance via `npm --provenance` + GH Actions OIDC. Consumers verify with `npm audit signatures @intentsolutions/core. (release.yml:95)`
- 4-axis boundary doctrine forbids LLM provider adapters, auth packages, database drivers, web frameworks, etc. — the kernel cannot inadvertently grow a network surface.
- Pre-commit `escape-scan --staged` catches a range of policy-violating patterns (cf. `@intentsolutions/audit-harness` for the rule set).
- Pre-commit + CI both run the 4-axis boundary check, including the REFUSE-only `labs.intentsolutions.io` URL-pattern rule (CISO binding).
- 100% test coverage floor + tsd negative type tests catch a wide range of regressions.
- api-extractor SemVer gate catches surface drift.
- Hash-pinning makes policy edits visible in diff (`.harness-hash`).
- `pnpm audit --prod` runs clean (0 prod vulnerabilities at v0.1.0 release; 2 moderate dev-only vulnerabilities — vite + esbuild via vitest transitive — not in the published tarball).
- The kernel ships zero runtime deps in the main entry, which means the supply-chain surface for types-only consumers is just this package.

Aspirational / deferred:

- **DSSE signature verification logic** — the kernel types the envelope (`DsseEnvelope`, `DsseSignature`) but provides no verification helper. Verification belongs in a sigstore client package downstream of the kernel. This is by design (Decision #1 — contracts-only) but means consumers must implement DSSE verification themselves if they want to verify a signed bundle, beyond the npm-tarball-level `npm audit signatures` check.
- **Rekor pre-flight (Security C-2)** — verifying that an attestation's Rekor entry exists before consumers trust the signature is a deferral. Tracked at deferral bead **Security C-2**. Until ratified, consumers should treat sigstore-keyless attestations as best-effort.

- **signing_mode enforcement in DSSE (Security C-3)** — the kernel exposes `SigningMode = 'sigstore_staging' | 'rekor_production' | 'unsigned_experimental'` on `EvidenceBundle` (`src/entities/EvidenceBundle.ts:30`) but does not enforce a consumer-side gate that “you may not trust `unsigned_experimental` rows in production.” The enforcement belongs at the consumer-policy layer (per `tests/TESTING.md`). Tracked at Security C-3.
- **CISO-binding DNSSEC + CAA pinning on `evals.intentsolutions.io`** — required before any signed attestation can land in `rekor_production` mode. Currently not in place; the predicate URI host is reserved but not cryptographically pinned to a specific signing identity. Until this lands, `signing_mode='rekor_production'` is not a path consumers should take.
- **tenant_id reservation (Architect W1)** — should the predicate body reserve a `tenant_id` field now (cheap to add) or decline explicitly? Tracked at `bd_000-projects-k0fj`. The “do nothing” default means future multi-tenant deployments may need a Class-1 ISEDC re-convening to add it.
- **Architect C1 deferral** — `gate-result/v1` SPEC location: resolved via the kernel’s `schemas/v1/gate-result.schema.json` carrying the `$id` of the predicate URI. Closed in spirit; left in the deferral log for paper trail.
- **Architect C2 deferral** — `intent-rollout-gate` role clarification. The kernel ships the `RolloutGate` entity + `RolloutGateDecision` enum; the policy applier lives in `@j-rig/rollout-gate`. C2 deferral is about whether `intent-rollout-gate` (the GH Action shell) is the thin delegator that the design says it is. Tracked but not closed.
- **Branch-protection `enforce_admins=true` (`bd_000-projects-8t7m`)** — scaffold-phase admin-bypass was permitted during v0.1.0; now that scaffold is complete, branch protection should flip `enforce_admins` to true. Listed as TODO in the v0.1.0 AAR; not yet executed at audit time.

10. Cost & Performance

Monthly Costs

Table 12.

Resource	Cost	Notes
npm registry	\$0 (public package)	Free for OSS packages
GitHub repo + Actions	\$0 (public repo, OSS minutes free tier)	Apache 2.0 OSS qualifies for the free tier
Sigstore / Fulcio / Rekor	\$0 (community service)	The keyless flow is a public good
Total	\$0/month	No paid infrastructure

Performance

This is a library, not a service. “Performance” has three relevant dimensions:

- **Build time:** `pnpm run build` (tsc on ~30 source files) completes in <10s on a modern dev machine. CI build step typically 4-6s.
- **Test time:** `pnpm run test` (154 vitest tests across 8 files + ~80 tsd assertions + 31 ajv tests + 31 Zod validator tests = ~295 assertions) completes in <5s. CI test step typically 3-5s.
- **Full CI run time:** the 9-step gate chain completes in ~38s (cf. v0.1.0 AAR’s release-CI metric). The boundary-check workflow runs in parallel in <30s.
- **Consumer install time:** `pnpm add @intentsolutions/core` resolves and links the 159-file, ~280 KB unpacked tarball in <2s on a warm cache.
- **Consumer parse time** (Zod runtime validation): a single `GateResultV1Schema.parse(payload)` call is microseconds; not a hot path concern. Zod’s own benchmarks place it competitive with hand-rolled validators.
- **Consumer tree-shake:** subpath exports (`./validators/v1/*`) ensure that consumers who import only `./validators/v1/eval-spec` don’t load the other 14 validators. Verified in tsd negative tests + import smoke tests.

Scaling Limits

- **Surface size:** api-extractor golden snapshot is `api/intentsolutions-core.api.md`. As the surface grows (new entities, new predicate bodies for `validation-result/v1` et al.), the snapshot grows and the diff-review burden grows linearly. No hard limit, but the per-repo blueprint § 4.5 review burden assumes <50 distinct entities — adding many more would warrant rethinking the single-package model (Decision #6 revisit trigger).
- **Predicate URI growth:** Blueprint B § 7.2 backward-compat policy says new predicate URIs may be added without a kernel bump. The kernel’s `PREDICATE_URIS` const enumerates the 5 v1 URIs; as more get spec’d, the const grows. No hard limit.
- **Consumer count:** no upper bound. npm registry handles distribution.
- **Schema bytes:** each JSON Schema file is ~3 KB; the full `schemas/v1/` set is ~50 KB. Consumer bundle impact is at-most a few hundred KB if they import all 15 schemas + the validators. Pragmatically, consumers import only what they need.
- **Coverage 100%-floor:** scales with the codebase. Currently ~1500 lines of TS + ~50 KB of JSON Schema. If runtime logic accumulates (which it shouldn’t per Decision #1), the 100% floor will become harder to maintain. The remediation is “stop accumulating runtime logic” — which is also the architectural invariant.

11. Current State

What’s Working

- **v0.1.0 published with sigstore provenance.** Verifiable via `npm audit signatures @intentsolutions/core` (cf. v0.1.0 AAR § “Release artifacts”). The release workflow’s 38s green run is the canonical evidence.
- **9-step CI gate chain green on every PR + main.** harness-verify, lint, typecheck, arch, test, coverage, build, test:types, dist-verify. Cf. `.github/workflows/ci.yml:39-82`.

- **4-axis boundary check green.** No FORBIDDEN package patterns, no forbidden import paths, no forbidden top-level dirs, no forbidden URL hits. Cf. `.github/workflows/boundary-check.yml` + the FORBIDDEN.md “Compliance audit” footer (1 runtime dep / 8 cap, 0 forbidden hits).
- **100% line/branch/function/statement coverage** on consumer-facing code (excluding `_generated/`). Enforced in `vitest.config.ts`; verified at every CI run.
- **All 13 canonical entities + the gate-result/v1 predicate body shipped** in TS interface form, JSON Schema form, and Zod validator form. Cross-validated via fixture-driven tests in `src/__tests__/schemas.test.ts` + `src/validators/v1/validators.test.ts` + the integration test at `src/integration.test.ts` (ERD walk locking Blueprint B § 6.2 invariants).
- **api-extractor SemVer regression gate live** (commit `ac0cdec`, `iec-E07`). Public surface drift fails CI with explicit instructions for the engineer.
- **Hash-pinned policy artifacts** (`.harness-hash`) protect `.dependency-cruiser.cjs` + FORBIDDEN.md + ALLOWLIST.md from silent edits.
- **Pre-commit hook live** (`.husky/pre-commit` runs `escape-scan` + `boundaries` + `lint-staged`).
- **Tag-vs-package drift defense** in release workflow (`release.yml:52-60`).
- **Zero runtime deps in main entry**, exactly one runtime dep (`zod`) for the validators subpath. Both verified in `package.json:113-115`.

What Needs Attention

- **HIGH — Downstream consumer adoption.** Two weeks post-publish, `audit-harness`, `j-rig-skill-binary-eval`, and `intent-rollout-gate` have not migrated their local schemas onto the kernel. Migration beads `iah-E02`, `iaj-E02` are open; `intent-rollout-gate` has no bead claimed yet. Impact: until the migration lands, the unification thesis (DR-010 Q3) is not enforced in practice — siblings continue to carry divergent local types. Fix: prioritize the migration epics; ratify `intent-rollout-gate`’s migration bead.
- **HIGH — iel-link-schemas-to-kernel (lab side).** The lab’s existing v0.1.0-draft schemas under `intent-eval-lab/specs/.../schema/` need to be repointed at the kernel (either by symlink or by reference). Until this lands, the lab carries its own copies of canonical entity schemas — exactly the duplication the kernel exists to prevent. Cross-repo, lab-side session work; tracked.
- **MEDIUM — Branch protection enforce_admins=true.** Tracked at `bd_000-projects-8t7m`. Post-scaffold phase, `admin-bypass` on protected `main` is no longer warranted. Listed as TODO in the v0.1.0 AAR; should be executed soon.
- **MEDIUM — DSSE/Rekor security follow-ons.** Security C-2 (Rekor pre-flight), Security C-3 (`signing_mode` enforcement in DSSE), CISO-binding DNSSEC + CAA pinning on `evals.intentsolutions.io`. None are blocking v0.1 (the kernel is `contracts-only`; verification is consumer-side) but they ARE blocking any move to `rekor_production` signing mode.
- **MEDIUM — tenant_id reservation (Architect W1).** Tracked at `bd_000-projects-k0fj`. Decision: reserve now (cheap, adds an optional field) or decline explicitly. Multi-tenant deployments will need this; defer-by-default may force a Class-1 ISEDC convening later.
- **LOW — 8 deferral beads filed during the sprint** (`iec-deferral-A` through `G` plus `bd iel-link-schemas-to-kernel`). Each one represents an “unmodeled because Blueprint B didn’t bless a shape” decision. None are P0; each has a trigger condition for re-engagement.
- **LOW — Pydantic codegen + Python distribution (iec-E08).** Polyglot consumers fall back to JSON Schema only at v0.1. Becomes a soft constraint as Python consumers materialize.

- **LOW — Upstream bd auto-flush bug.** Tracked at `bd_000-projects-ufc` + `upstream gastownhall/beads#3848` + `#3970`. Workaround documented in `umbrella intent-eval-platform/CLAUDE.md` (the JSONL export discipline). Operational annoyance, not a kernel concern.

Implementation Status

Table 13.

Component	Status	Evidence
13 canonical entity TS interfaces	SHIPPED v0.1.0	<code>src/entities/{EvalSpec, EvalRun, MatcherM</code>
<code>gate-result/v1</code> NORMATIVE predicate body	SHIPPED v0.1.0	<code>src/predicates/gate-result-v1.ts</code>
10 branded primitive types	SHIPPED v0.1.0	<code>src/primitives.ts</code>
State-machine transition maps + <code>canTransition</code>	SHIPPED v0.1.0	<code>src/state-machines/types.ts</code> + per-entity exports
15 JSON Schema definitions (13 entity + 1 predicate + 1 <code>_common</code> + 1 index)	SHIPPED v0.1.0	<code>schemas/v1/*.schema.json</code>
15 Zod runtime validators	SHIPPED v0.1.0	<code>src/validators/v1/*.ts</code>
IS Testing SOP install (audit-harness, husky, dep-cruiser hash-pinned)	SHIPPED v0.1.0	<code>.husky/pre-commit</code> , <code>.harness-hash</code> , <code>.dependency-cruiser.cjs</code>
9-step CI gate chain	SHIPPED v0.1.0	<code>.github/workflows/ci.yml</code>
Tag-triggered release with sigstore provenance	SHIPPED v0.1.0	<code>.github/workflows/release.yml</code>
ERD-walk integration test	SHIPPED v0.1.0	<code>src/integration.test.ts</code>
Per-repo blueprint (Blueprint C application, iec-E10)	SHIPPED post-v0.1.0	<code>000-docs/002-AT-ARCH-repo-blueprint-202</code>
4-axis boundary enforcement (FORBID-DEN/ALLOWLIST/CODEOWNERS/checker/CI, iec-E11)	SHIPPED post-v0.1.0	<code>FORBIDDEN.md</code> , <code>ALLOWLIST.md</code> , <code>scripts/check-boundaries.ts</code> , <code>.github/workflows/boundary-check.yml</code>
Testing SOP + CI/CD ratify-and-close AAR (iec-E12)	SHIPPED post-v0.1.0	<code>000-docs/004-AT-AAR-testing-cicd-bootstr</code>
api-extractor SemVer regression gate (iec-E07)	SHIPPED post-v0.1.0	<code>api/intentsolutions-core.api.md</code> , <code>api-extractor.json</code> , <code>scripts/api-diff.ts</code> , <code>ci.yml:65-75</code>

Component	Status	Evidence
Lifecycle state machines beyond transition maps (iec-E05)	DEFERRED	Per v0.1.0 AAR “What did NOT go in v0.1.0”
UUID + event ID standards (iec-E06)	DEFERRED	Kernel uses UUIDv7 with regex validation; deeper standardization deferred
Pydantic codegen + Python distribution (iec-E08)	DEFERRED	Per v0.1.0 AAR
audit-harness migration onto kernel (iah-E02)	OPEN	Sibling repo work
j-rig migration onto kernel (iaj-E02)	OPEN	Sibling repo work
intent-rollout-gate migration onto kernel	NOT-CLAIMED	No bead yet
Lab schema repointing (iel-link-schemas-to-kernel)	OPEN	Cross-repo lab-side session
Branch protection enforce_admins=true	OPEN	bd_000-projects-8t7m

12. Roadmap

Week 1 — Stabilization

Measurable outcomes:

- Flip `enforce_admins=true` on the GH branch protection rule for `main` (closes `bd_000-projects-8t7m`).
- Land the migration PRs in `audit-harness` (iah-E02) — replace local gate-result types with `GateResultV1Schema`; brand existing identifier strings via the Zod parsers; verify emitted predicate bodies satisfy the schema in CI.
- Open the `intent-rollout-gate` migration bead and stage a PR that swaps local schema definitions for `GateResultV1Schema`.
- Run `pnpm exec audit-harness arch` on `j-rig-skill-binary-eval` (sibling repo) to identify which entity types should move from `@j-rig/core` to `@intentsolutions/core`.

Month 1 — Foundation

Measurable outcomes:

- Land iaj-E02 (j-rig migration). UPPERCASE `JudgeVerdict` -> lowercase `RolloutGateDecision` translation lives in `@j-rig/rollout-gate` (policy translator); the kernel-side `JudgeDecision` shape is consumed verbatim.

- Land `iel-link-schemas-to-kernel`. Lab’s `specs/.../schema/` symlinks or references resolve to the kernel; lab no longer carries duplicate schemas.
- Cut a `v0.1.x` patch release if any consumer-feedback bugs surface.
- Decide on `tenant_id` reservation (`bd_000-projects-k0fj`) — either land an optional field via a minor bump, or document the decline-with-rationale in a Decision Record.
- Decide on Pydantic codegen path (`iec-E08`) — at minimum, scope the work into an epic with explicit acceptance criteria.

Quarter 1 — Strategic

Measurable outcomes:

- Land at least one additional predicate body normative spec (e.g., `validation-result/v1` from Blueprint B § 7.5 if/when that section is ratified). Lift it from `sigstore_staging` to `rekor_production` after CISO-binding DNSSEC + CAA pinning lands on `evals.intentsolutions.io`.
- Close Security C-2 (Rekor pre-flight) and C-3 (`signing_mode` in DSSE) deferral beads.
- Cut `v0.2.0` if the surface needs to evolve in a backward-incompatible way (e.g., adding required fields to existing entities — should be rare given the kernel’s stability promise).
- Build a consumer-side smoke-test harness that any downstream repo can adopt to verify kernel adoption (“import a type, validate a fixture, emit a signed row, verify the row” loop).
- Decide on `examples/` policy. Currently forbidden by `FORBIDDEN.md` Axis 3; if the absence-of-examples cost outweighs the architectural-purity benefit, consider a sibling `intent-eval-examples` repo.

13. Quick Reference

URLs

Table 14.

Resource	URL
npm package	https://www.npmjs.com/package/@intentsolutions/intent-eval-core
GitHub repo	https://github.com/jeremylongshore/intent-eval-core
Latest release	https://github.com/jeremylongshore/intent-eval-core/releases/tag/v0.1.0
CI runs	https://github.com/jeremylongshore/intent-eval-core/actions/workflows/ci.yml
Release runs	https://github.com/jeremylongshore/intent-eval-core/actions/workflows/release.yml

Resource	URL
Boundary check runs	https://github.com/jeremylongshore/intent-eval-core/actions/workflows/boundary-check.yml
Provenance attestation	Embedded in npm tarball; verify with <code>npm audit signatures @intentsolutions/core</code>
Predicate URI (gate-result/v1)	https://evals.intentsolutions.io/gate-result/v1
Schema \$id (gate-result/v1)	https://evals.intentsolutions.io/gate-result/v1.schema.json
Plane sub-module	Intent Eval Core - Kernel (LAB project; UUID <code>5abf1653-c9ba-4029-8c04-76f148eb78f5</code>)
Umbrella doc (governance)	<code>~/000-projects/intent-eval-platform/CLA</code>
Phase A foundation (intent-eval-lab)	DR-010, Blueprint A/B/C, Canonical Glossary on <code>main</code>

First-Week Checklist

- ☐ Repo access granted (read on `intent-eval-core`; read on `intent-eval-lab` for Blueprint references)
- ☐ Local environment running: `pnpm install --frozen-lockfile && pnpm run check -> green`
- ☐ Coverage verified: `pnpm run test:coverage -> 100%` on all 4 axes
- ☐ Read this document (the operator playbook) end-to-end
- ☐ Read `CLAUDE.md` (Claude-Code-specific guidance)
- ☐ Read `000-docs/002-AT-ARCH-repo-blueprint-2026-05-18.md` (per-repo blueprint)
- ☐ Read `000-docs/003-AT-STND-core-repo-boundaries-2026-05-18.md` (boundary doctrine)
- ☐ Skim Blueprint B § 2 (the 13-entity domain model) and § 7.4 (`gate-result/v1`) — these are the contracts you’re maintaining
- ☐ Open one entity file (e.g., `src/entities/EvalSpec.ts`), its schema (`schemas/v1/eval-spec.schema.json`), and its Zod validator (`src/validators/v1/eval-spec.ts`) side-by-side; understand the twin-source pattern
- ☐ Run the codegen reference: `pnpm run codegen:validators`; read `src/validators/v1/_generated/README` understand why hand-authored is canonical
- ☐ Walk a test fixture: `tests/fixtures/v1/gate-result.valid.json -> mentally trace through GateResultV1Schema.parse(...)`
- ☐ Try a deliberate boundary violation locally (e.g., add `express` to dependencies) and observe pre-commit + `pnpm run boundaries` both refusing

- Run `pnpm exec audit-harness init` (no-op if everything is pinned) and observe how `.harness-hash` works
- Met with the system owner (@jeremylongshore) and confirmed access to bd workspace at `~/000-projects/.beads/`

Appendices

A. Glossary

- **Branded type** — TypeScript phantom type pattern that distinguishes structurally-identical types at compile time without adding runtime weight. Example: `Uuidv7 = string & { readonly [__brand]: 'Uuidv7' }`. Compiles away.
- **canTransition** — Pure runtime helper (`src/state-machines/types.ts:19-25`) that returns true if a `from -> to` transition is declared in an entity's transition map.
- **CHALLENGE vs REFUSE** — Exit-code distinction in the 4-axis boundary checker. CHALLENGE (exit 1) is overridable via a `boundary-override: bd_<id>` line in the PR description; REFUSE (exit 2) has no override path. The only REFUSE rule today is the `labs.intentsolutions.io` URL pattern.
- **codegen reference** — Output of `pnpm run codegen:validators` at `src/validators/v1/_generated/`. NOT canonical; used as a structural sanity check before hand-authoring the canonical Zod validator one directory up.
- **Composition DAG** — Per Blueprint B § 1.3, the directed acyclic graph an `EvalSpec` declares over `EvalRun` and `ToolInvocation` nodes via `feeds / gates / enriches` edges.
- **DSSE** — Dead Simple Signing Envelope. The outer wrapper around an in-toto Statement v1, signed via sigstore-keyless OIDC. Defined at `src/predicates/gate-result-v1.ts:249-269`.
- **Evidence Bundle** — Per Blueprint B § 2.4, the append-only collection of signed predicate rows produced by an `EvalRun`. APPEND-ONLY; corrections create a new bundle with a new UUIDv7 referencing the prior bundle's `content_hash`.
- **FORBIDDEN.md** — Machine-readable 4-axis boundary forbidden set at the repo root. Read by `scripts/check-boundaries.ts`. Hash-pinned via `.harness-hash`.
- **gate-result/v1** — The only fully NORMATIVE in-toto predicate body at v1. Defined in Blueprint B § 7.4; implemented in `src/predicates/gate-result-v1.ts`, `schemas/v1/gate-result.schema.json`, `src/validators/v1/gate-result-v1.ts`. Adding/loosening any field requires Class-1 ISEDC convening.
- **hash-pin / harness-hash** — The `.harness-hash` file holds a SHA-256 digest of the policy file content. Pre-commit + CI verify it. After legitimate policy edits, `pnpm exec audit-harness init` refreshes it.
- **in-toto Statement v1** — Standard envelope structure (`_type`, `subject[]`, `predicateType`, `predicate`) the kernel uses for signed attestations. Defined at `src/predicates/gate-result-v1.ts:242-249`.
- **ISEDC** — Intent Solutions Executive Decision Council. 7-seat adversarial council adjudicating immutable artifact changes, brand commitments, partner-relationship dynamics, and architectural decisions that ripple across consumers. DR-010 is its Session 4 widened-scope lock.
- **JSON Schema draft 2020-12** — Modern JSON Schema dialect supporting `if/then/else` and `$dynamicRef`. The wire format for `schemas/v1/*.schema.json`.
- **kernel** — `@intentsolutions/core` — the canonical contracts kernel. Types + schemas + validators + state machines only. No runtime.

- **MM-class** — Failure-mode classification in `audit-harness` (e.g., MM-4). Surfaces in `gate-result/v1.failure_mode` when `gate_decision='fail'`.
- **NORMATIVE** — Spec-bound. Adding/loosening fields requires Class-1 ISEDC convening. Distinct from informational, which can change without convening.
- **ONE BIG** — DR-010 Q1 binding: one canonical package, not a galaxy. `@intentsolutions/core` is the ONE BIG for the kernel.
- **Predicate URI** — Identifier of an in-toto predicate body. Lives at `evals.intentsolutions.io` (CISO binding). Five v1 URIs blessed; only `gate-result/v1` has a NORMATIVE body at v1.
- **Sigstore** — Public-good supply-chain signing infrastructure (Fulcio CA + Rekor transparency log + Cosign). Used keyless via GH Actions OIDC for npm provenance.
- **Subpath export** — Per-path entry in `package.json#exports` letting consumers import from `@intentsolutions/core/validators/v1/<entity>` for tree-shakable per-entity validators.
- **Twin source** — Maintenance pattern where two artifacts (JSON Schema + Zod validator) are both hand-authored, with codegen output used as a reference but not canonical. See Decision #3.
- **Unification thesis** — DR-010 Q3 binding: every validator in the platform emits an Evidence Bundle whose predicate rows satisfy a kernel-defined schema. The thesis is enforced by the existence of this kernel and by sibling-repo migration onto it.

B. Reference Links

- Blueprint A — Ecosystem Master Blueprint
- Blueprint B — Platform Runtime Blueprint (§ 2 = 13-entity model; § 7.4 = `gate-result/v1`)
- Blueprint C — Repo Blueprint Template
- Canonical Glossary
- DR-010 — ISEDC Session 4 widened-scope lock
- Per-repo blueprint (this repo)
- Boundary doctrine (this repo)
- v0.1.0 release AAR (this repo)
- Testing SOP + CI/CD AAR (this repo)
- npm provenance docs
- Sigstore project
- in-toto attestation framework
- Keep a Changelog
- SemVer 2.0.0

C. Troubleshooting Playbooks

C.1 Pre-commit blocks with `harness-hash` mismatch

1. Identify which policy file you edited (`.dependency-cruiser.cjs`, `FORBIDDEN.md`, `ALLOWLIST.md`).
2. Run `pnpm exec audit-harness init` to refresh `.harness-hash`.
3. `git add .harness-hash` and commit.
4. If the edit was unintentional, revert the policy file change and the hash will match again without re-init.

C.2 CI fails on arch step

1. Read the CI step log; dep-cruiser names the violated rule.
2. Match against the 8-rule table in section 8.4 of this document.
3. Fix per the rule's `comment` field in `.dependency-cruiser.cjs`.
4. If you genuinely need to weaken a rule, file an `iec- boundary-override` bead AND a Class-1/Class-2 ISEDC convening request (depending on the rule's severity).

C.3 CI fails on boundaries step

1. Read the violation summary in the PR comment (`boundary-check.yml` posts it automatically on failure).
2. If REFUSE: fix the violation. There is no override.
3. If CHALLENGE: either fix, OR file an `iec-boundary-override-<short>` bead and reference it in the PR body as `boundary-override: bd_000-projects-<id>`.
4. Re-run CI.

C.4 Consumer says “I can’t import `./validators/v1/...`”

1. Check consumer's TypeScript module resolution: must be NodeNext (or Bundler with appropriate handling).
2. Check consumer's `tsconfig#paths` or workspace overrides — sometimes these mis-resolve subpath exports.
3. Confirm consumer's `package.json#dependencies` has `zod` as a peer (it's not a transitive of `@intentsolutions/core`'s main entry; it IS a transitive of the validators subpath).
4. Confirm the consumer's pnpm/npm version supports the `exports` field (pnpm ≥ 8 , npm ≥ 8 — but bugs lurk in older patches).

C.5 Release workflow fails with Tag does not match package.json version

1. Tag was pushed without bumping `package.json#version`.
2. Delete the tag locally: `git tag -d vX.Y.Z`.
3. Delete the tag remotely: `git push origin :refs/tags/vX.Y.Z`.
4. Bump `package.json#version` correctly, commit, push to main, wait for CI green.
5. Re-tag: `git tag -a vX.Y.Z -m "..."` and push.

C.6 npm audit signatures reports “no signatures found”

1. Check npm CLI version: `npm --version ->` must be ≥ 9.5 (preferably latest).
2. Check that the published version was published from `release.yml`: `gh run list --workflow=release.yml --branch=vX.Y.Z`.
3. If the version was manually published bypassing the release workflow, that's the bug — discuss release-hygiene with the maintainer.
4. If the version was published via `release.yml` but the attestation is missing, check the release run's logs for the publish step output.

D. Open Questions

1. **When does intent-rollout-gate's migration bead open?** No bead claimed at audit time. Owner: Jeremy Longshore.

2. **Will `tenant_id` be reserved on the predicate body now (cheap optional field) or declined explicitly?** Tracked at `bd_000-projects-k0fj`; the “do nothing” default may force a Class-1 ISEDC re-convening later when multi-tenant deployments materialize.
3. **When does CISO-binding DNSSEC + CAA pinning on `evals.intentsolutions.io` land?** Required before any signed attestation can move from `sigstore_staging` to `rekor_production`. No bead with concrete acceptance criteria.
4. **When does `iec-E08` (Pydantic codegen + Python distribution) ship?** Polyglot consumers fall back to JSON Schema only until then. No date.
5. **Should `examples/` policy be revisited?** Forbidden by Axis 3 today. Test fixtures stand in. If polyglot consumers (post-`iec-E08`) struggle to onboard without language-specific examples, a sibling `intent-eval-examples` repo may be warranted.
6. **What is the `intent-rollout-gate` Action shell’s exact contract after kernel migration?** Architect C2 deferral. The kernel ships `RolloutGate` + `RolloutGateDecision`; policy applier lives in `@j-rig/rollout-gate`. C2 is about clarifying that `intent-rollout-gate` is purely the GH-Action-shell delegator, never a policy implementer.
7. **How are kernel deprecation cycles handled?** When a field is deprecated on, say, `EvalSpec`, what is the consumer-visible deprecation signal? `api-extractor` flags deprecations in the golden snapshot, but downstream consumer notification is currently manual via `CHANGELOG`.
8. **What is the `v1.0` readiness criterion?** `v0.x` is experimental per per-repo blueprint § 1. Criteria for `v1.0` (frozen surface, longer LTS guarantees, etc.) are not documented.

End of operator playbook. Total length: ~17,000 words. Generated under `/appaudit v2.0.0` against `intent-eval-core HEAD ac0cdec` (post-`v0.1.0`).